arXiv:2601.09881v2 [cs.CV] 9 Jul 2026

Transition Matching Distillation for Fast Video Generation

Weili Nie[1*], Julius Berner[1*], Nanye Ma[2], Chao Liu[1], Saining Xie[2], Arash Vahdat[1]
[1]NVIDIA [2]NYU [*]equal contribution

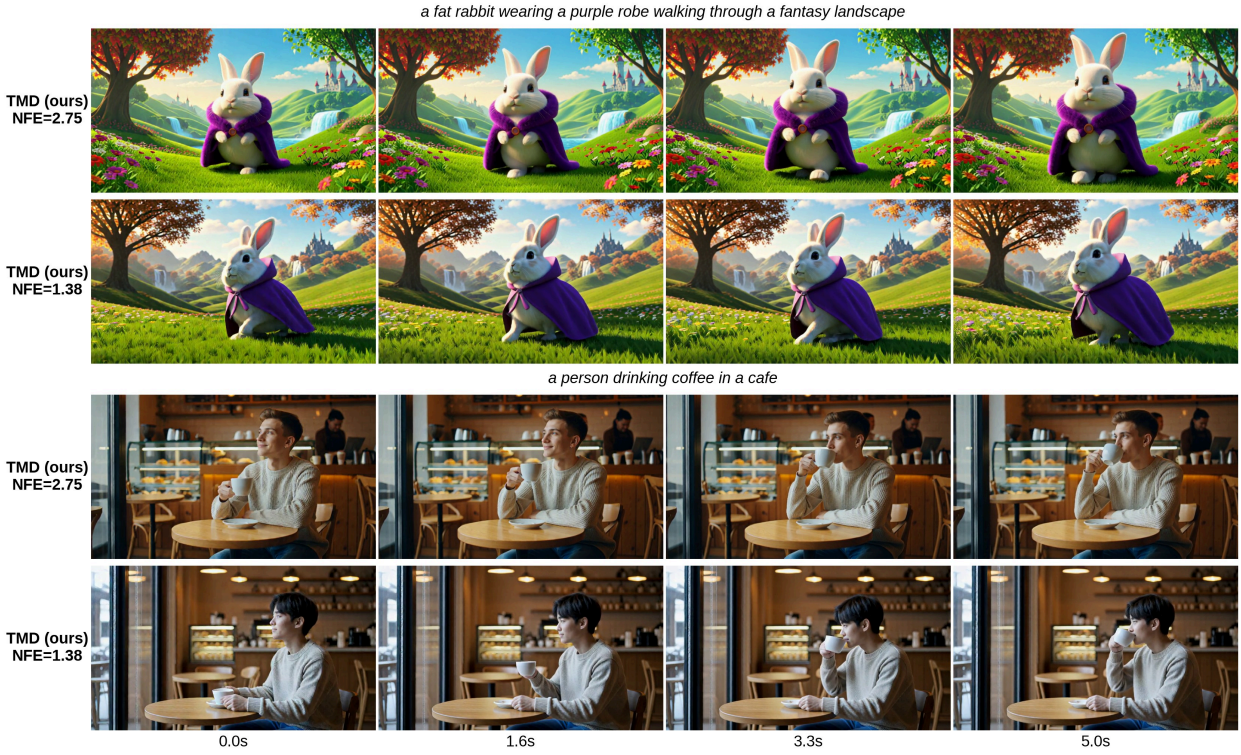


Figure 1. **Generated examples from TMD.** Four frames of 5s 480p videos generated from two text prompts using our TMD method (distilled from Wan2.1 14B T2V) with two different (effective) number of function evaluations (NFE).

Abstract

Large video diffusion and flow models have achieved remarkable success in high-quality video generation, but their use in real-time interactive applications remains limited due to their inefficient multi-step sampling process. In this work, we present Transition Matching Distillation (TMD), a novel framework for distilling video diffusion models into efficient few-step generators. The central idea of TMD is to match the multi-step denoising trajectory of a diffusion model with a few-step probability transition process, where each transition is modeled as a lightweight conditional flow. To enable efficient distillation, we decompose the original diffusion backbone into two components: (1) a main backbone, comprising the majority of early layers, that extracts semantic representations at each outer transition step; and (2) a flow head, consisting of the last few layers, that leverages these representations to perform multiple inner flow updates. Given a pretrained video flow model, we first introduce a flow head to the model, and adapt it into a conditional flow map. We then apply distribution matching distillation to the student model with flow head rollout in each transition step. Extensive experiments on distilling Wan2.1 1.3B and 14B text-to-video models demonstrate that TMD provides a flexible and strong trade-off between generation speed and visual quality. In particular, TMD outperforms existing distilled models under comparable inference costs in terms of visual fidelity and prompt adherence.

1. Introduction

Recent progresses in large-scale diffusion models [19, 45] have significantly advanced the frontier of video generation [1, 5, 15, 25, 49, 57]. Open-sourced models (such as

HunyuanVideo [25], Wan [49] and Cosmos [1]) and commercial text-to-video (T2V) systems (such as Sora, Veo and Kling) demonstrate remarkable capabilities in synthesizing coherent and photorealistic videos from text prompts. Despite their success, sampling inefficiency remains a central bottleneck. Standard diffusion models rely on a multi-step denoising process, often requiring hundreds of iterative steps, to progressively transform noise into realistic outputs [25, 49]. This iterative nature leads to high inference latency and computational cost, rendering large diffusion models impractical for interactive applications such as real-time video generation, content editing, or world modeling for agent training. Accelerating diffusion sampling without sacrificing visual quality becomes a key open challenge.

A growing body of research has explored diffusion distillation to compress long denoising trajectories into a small number of inference steps. Existing approaches can be broadly categorized into two families: (1) *trajectory-based distillation*, which includes knowledge distillation [36, 40] and consistency models [16, 17, 35, 46] that directly regress the teacher's denoising trajectories; and (2) *distribution-based distillation*, encompassing adversarial [41, 42] and variational score distillation [58, 59, 70] methods that align the student and teacher distributions. These techniques can reduce the sampling process to as few as one or two steps in the image domain. However, extending them to video diffusion models presents unique challenges. Videos exhibit high spatiotemporal dimensionality and complex inter-frame dependencies, making it difficult to preserve both global motion coherence and fine-grained spatial details during distillation. Most existing methods treat the diffusion network as a monolithic mapping [28, 64, 69], neglecting the hierarchical structure and semantic progression inherent in large video diffusion backbones.

To address these limitations, we propose *Transition Matching Distillation* (TMD) to distill large video diffusion models into few-step generators (*e.g.*, less than 4 steps; see Figure 1 for an example). Inspired by *Transition Matching* [44], TMD approximates the many-step denoising process with a compact few-step probability transition process, where each transition captures the distributional evolution of video samples across widely separated noise levels, enabling the student to take large transition steps that match the teacher model's distribution. To model the transition process, we introduce a decoupled architecture for the student model with two components: (1) a main backbone, comprising the majority of early layers, that extracts high-level semantic representations at each outer transition step; and (2) a flow head, consisting of the final few layers, that conditions on these representations and refines the fine-grained visual details through multiple inner flow updates.

This hierarchical decomposition allows the student to share representations from the main backbone with the flow head. The flow head performs a few lightweight inner refinement steps within each outer transition step, providing a flexible mechanism to balance sampling efficiency and visual fidelity. First, we pretrain the student via trajectory-based distillation for the flow head using an adaption of *MeanFlow* [17]. Then, we formulate distillation as a distribution matching problem between the teacher denoising process and student transition process using an improved version of *DMD2* [58]. By unrolling flow head during training, we align the student's probability transitions with the teacher's multi-step diffusion distribution, capturing both semantic evolution and fine-grained visual details.

We evaluate TMD in distilling Wan2.1 1.3B and 14B T2V models on VBench [22] and a user preference study. Our experiments show TMD consistently outperforms existing distilled methods under comparable inference budgets, achieving better visual fidelity and prompt adherence. Particularly, our distilled 14B model achieves an overall score of 84.24 on VBench in near-one-step generation (NFE=1.38). In summary, our main contributions include:

- **A novel distillation framework for video diffusion**: We propose Transition Matching Distillation (TMD), which distills long denoising trajectories into a compact few-step probability transition process.
- **A decoupled diffusion backbone design**: We decompose the teacher model into a semantic backbone and a recurrent flow head, enabling hierarchical distillation with flexible inner flow refinement.
- **A two-stage training strategy**: (1) transition matching adaption that converts the flow head into a conditional flow map and (2) distribution matching distillation with flow head rollout in each transition step.
- **Comprehensive empirical validation**: We demonstrate the effectiveness of TMD in distilling Wan2.1 1.3B and 14B T2V models, achieving state-of-the-art trade-offs between speed and quality in few-step video generation.

2. Background

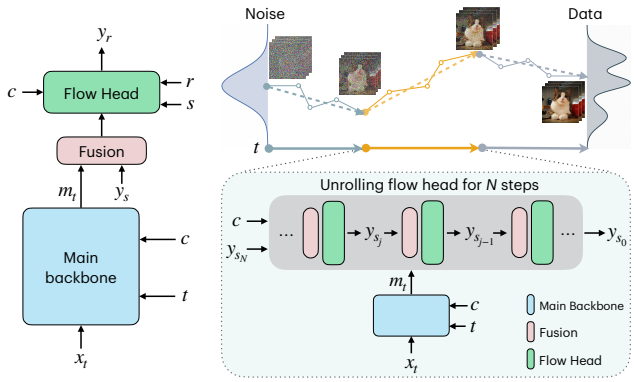
Transition Matching. *Transition matching* (TM) [44] is a continuous-state generative model, that can be viewed as a generalization of flow matching to discrete time. It uses the rectified flow schedule

$$\boldsymbol{x}_t = (1-t)\boldsymbol{x} + t\boldsymbol{x}_1, \quad t \in [0,1], \tag{1}$$

interpolating between the data $\boldsymbol{x}$ and a standard normal $\boldsymbol{x}_1 \sim \mathcal{N}(0, I)$. Flow matching approximates the *instantaneous velocity*

$$\boldsymbol{v}(\boldsymbol{x}, t) := \mathbb{E}[\dot{\boldsymbol{x}}_t | \boldsymbol{x}] = \mathbb{E}[\boldsymbol{x}_1 - \boldsymbol{x} | \boldsymbol{x}], \tag{2}$$

requiring many small time-steps to generate data $\boldsymbol{x}$ from noise $\boldsymbol{x}_1$ [2, 32, 34]. In contrast, TM directly models the probabilistic transition between two states $\boldsymbol{x}_{t_i}$ to $\boldsymbol{x}_{t_{i-1}}$,



(a) Decoupled architecture (b) Transition process with flow head rollout

Figure 2. **Overview of our TMD method.** (a) Decoupled architecture for TMD student, where the main backbone takes the noisy sample $\boldsymbol{x}_t$, timestep t and text conditioning c as inputs and outputs the main feature $\boldsymbol{m}_t$, and with a simple fusion layer, flow head conditions on $\boldsymbol{m}_t, c$ and predicts the less noisy target $\boldsymbol{y}_r$ from the more noisy $\boldsymbol{y}_s$ ($s \geq r$). (b) Top: Transition process maps noise to data with a few transition steps. Bottom: In each step, flow head rollout is performed during both distillation and sampling. We omit the timestep inputs s and r to the flow head for simplicity.

where $0 = t_0 < t_1 < \cdots < t_M = 1$ is a given time discretization. For every transition the model is learned to predict an auxiliary latent variable $\boldsymbol{y}$, such that $\boldsymbol{x}_{t_{i-1}}$ is easy to sample (*e.g.*, deterministic) given $\boldsymbol{y}$ and $\boldsymbol{x}_{t_i}$. For instance, *Difference Transition Matching* (DTM) considers[1]

$$\boldsymbol{y} \coloneqq \boldsymbol{x}_1 - \boldsymbol{x} \tag{3}$$

such that

$$\boldsymbol{x}_{t_{i-1}} = \boldsymbol{x}_{t_i} - (t_i - t_{i-1})\boldsymbol{y}. \tag{4}$$

To predict $\boldsymbol{y}$ given $\boldsymbol{x}_{t_i}$, transition matching considers an "inner" rectified flow schedule

$$\boldsymbol{y}_s = (1-s)\boldsymbol{y} + s\boldsymbol{y}_1, \quad s \in [0,1], \tag{5}$$

where $\boldsymbol{y}_1 \sim \mathcal{N}(0, I)$. In practice, at every step $\boldsymbol{x}_{t_i}$, the main backbone predicts features and a lightweight head is learned using flow matching to approximate the inner velocity

$$\boldsymbol{v}(\boldsymbol{y}, s; \boldsymbol{x}_{t_i}) \coloneqq \mathbb{E}[\dot{\boldsymbol{y}}_s | \boldsymbol{y}, \boldsymbol{x}_{t_i}] = \mathbb{E}[\boldsymbol{y}_1 - (\boldsymbol{x}_1 - \boldsymbol{x}) | \boldsymbol{y}, \boldsymbol{x}_{t_i}] \tag{6}$$

While DTM provided improved performance in image generation, it still needs around 30 transition steps in sampling.

MeanFlow. To accelerate the sampling of diffusion models, MeanFlow [17] proposes to learn a *flow map* $\boldsymbol{f}(\boldsymbol{y}_s, s, r)$ [4], which maps the point $\boldsymbol{y}_s$ at arbitrary time s on the PF-ODE trajectory to any preceding point x_r for $r < s$. Formally,

$$\boldsymbol{f}(\boldsymbol{y}_s, s, r) \coloneqq \boldsymbol{y}_s - \int_s^r \boldsymbol{v}(\boldsymbol{y}_\tau, \tau)\, \mathrm{d}\tau \tag{7}$$

where $\boldsymbol{v}$ denotes the instantaneous velocity as in (6). MeanFlow parameterizes this mapping using the *average velocity* along the trajectory segment from $\boldsymbol{y}_s$ to $\boldsymbol{y}_r$, expressed as $\boldsymbol{f}(\boldsymbol{y}_s, s, r) = \boldsymbol{y}_s + (s-r)\boldsymbol{u}(\boldsymbol{y}_s, s, r)$, where $\boldsymbol{u}$ represents the average velocity. Combining with (7), we have $(s-r)\boldsymbol{u}(\boldsymbol{y}_s, s, r) = -\int_s^r \boldsymbol{v}(\boldsymbol{y}_\tau, \tau)\, \mathrm{d}\tau$. This inspires the key insight from MeanFlow, where differentiating both sides w.r.t s obtains the following identity:

$$\boldsymbol{u}(\boldsymbol{y}_s, s, r) + (s-r)\frac{\mathrm{d}}{\mathrm{d}s}\boldsymbol{u}(\boldsymbol{y}_s, s, r) = \boldsymbol{v}(\boldsymbol{y}_s, s), \tag{8}$$

where $\frac{\mathrm{d}}{\mathrm{d}s}\boldsymbol{u}(\boldsymbol{y}_s, s, r)$ refers to the total derivative. This identity motivates the following practical training objective:

$$\mathcal{L}(\theta) \coloneqq \mathbb{E}_{s,r,\boldsymbol{y}_s}\left[\|\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r) - \hat{\boldsymbol{u}}\|^2\right] \tag{9}$$

with

$$\hat{\boldsymbol{u}} \coloneqq \mathrm{sg}\big(\boldsymbol{v}(\boldsymbol{y}_s, s) - (s-r)\tfrac{\mathrm{d}}{\mathrm{d}s}\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r)\big), \tag{10}$$

where $\mathrm{sg}(\cdot)$ refers to the stop gradient operator, the instantaneous velocity $\boldsymbol{v}(\boldsymbol{y}_s, s)$ is approximated by conditional velocity $\boldsymbol{y}_1 - \boldsymbol{y}$, and the total derivative $\frac{\mathrm{d}}{\mathrm{d}s}\boldsymbol{u}(\boldsymbol{y}_s, s, r)$ can be computed using either forward-mode automatic differentiation or a finite-difference approximation [47, 52].

DMD. Flow maps like MeanFlow must learn mappings between points along the ODE trajectory, which is difficult to scale to video generation due to high dimensionality and large trajectory curvature (see Appendix B). In contrast, distribution distillation methods such as DMD2 [58] offer more flexibility by matching only the output distribution to that of the teacher or data—using a GAN [18] loss for the latter and a variational score distillation (VSD) [59] loss, i.e., reverse KL divergence, for the former. This can be represented as

$$\mathcal{L}(\theta) = \mathbb{E}_{t_i, \boldsymbol{x}_{t_i}, t, \hat{\boldsymbol{x}}_t}\left[w(t)\mathrm{sg}\big(D(\hat{\boldsymbol{x}}_t, t)\big)^T \hat{\boldsymbol{x}}\right], \tag{11}$$

where $\hat{\boldsymbol{x}} = \boldsymbol{g}_\theta(\boldsymbol{x}_{t_i}, t_i)$ denotes the student output from the noisy input $\boldsymbol{x}_{t_i}$, and t_i is sampled from a given student time discretization, w is a weighting function, D is the difference between the scores of the student and teacher distributions, $\hat{\boldsymbol{x}}_t = (1-t)\,\hat{\boldsymbol{x}} + t\boldsymbol{x}_1$ is a noisy sample from the forward process in (1) that is passed to score functions.

Since we do not have access to the score of the student distribution, a so-called *fake score* is initialized with the teacher parameters and trained on data from the student using flow matching. Up to a time-dependent weighting, D equals the difference between the *velocities* of the teacher and fake score. In practice, both the fake score and GAN discriminator are updated for a given number of iterations in between updates of the student and the discriminator consists of a lightweight head operating on intermediate features of the fake score or teacher.

3. Method

We introduce our method – *Transition Matching Distillation* (TMD), including two training stages: (1) *transition matching pretraining* to initialize a flow head capable of

[1] While TM uses a probabilistic model to predict $\boldsymbol{y}$, flow matching predicts the conditional expectation of $\boldsymbol{y}$ (Eq. (2)).

iteratively refine features extracted from the main backbone; and (2) *distillation with flow head* where we introduce DMD2-*v* that advances DMD2 in few-step video generation and apply it with flow head rollout at each transition step. For ease of presentation, we drop additional conditioning of the teacher model, such as text conditioning, in our notation. Below, we start with introducing our student architecture before presenting our two-stage training.

Decoupled architecture. Our method follows the general formulation of transition matching explained in Section 2. Different from TM, we aim to approximate many small denoising steps of a teacher model with the large transition step of the distilled student. To efficiently predict $\boldsymbol{y}$ in every transition step t_i, we decouple the pretrained teacher architecture into a main backbone $\boldsymbol{m}_\theta$, that acts as a feature extractor, and a lightweight flow head $\boldsymbol{f}_\theta$ that iteratively predicts $\boldsymbol{y}$ given these features, i.e.,

$$\boldsymbol{y}_{s_{j-1}} \approx \boldsymbol{f}_\theta\big(\boldsymbol{y}_{s_j}, s_j, s_{j-1}; \boldsymbol{m}_\theta(\boldsymbol{x}_{t_i}, t_i)\big), \tag{12}$$

where $0 = s_0 < s_1 < \cdots < s_N = 1$ is a given time discretization for the inner flow; see also Figure 2.

While such decoupling has been successfully used for training generative models [27, 44, 50, 61, 67], it requires a careful design to minimally disrupt the pretrained model. Our design considers two key factors: (1) *flow head target* $\boldsymbol{y}$. We find that the DTM formulation $\boldsymbol{y} = \boldsymbol{x}_1 - \boldsymbol{x}$ outperforms other target types, such as the sample prediction $\boldsymbol{y} = \boldsymbol{x}$ (see Appendix B). (2) *fusion layer*. We use a time-conditioned gating mechanism to fuse the main feature $\boldsymbol{m}_{t_i}$ and the noisy flow head target $\boldsymbol{y}_{s_j}$, ensuring the student's initial forward pass matches that of the teacher. See Appendix A for details, where we also provide pseudo-code for our inference in Algorithm 1.

3.1. Stage 1: Transition matching pretraining

Given the decoupled architecture, we convert the flow head into a flow map for iterative refinement before distillation. Similar to TM, we can directly use a flow matching loss in Eq. (6) to train the flow head to approximate the velocity of the inner flow. However, in theory, this would still require many inner steps to approximate $\boldsymbol{y}$. Thus, we leverage MeanFlow to obtain a few-step flow head.

Transition matching MeanFlow. On a high-level, our pretraining algorithm, termed *Transition Matching MeanFlow* (TM-MF), uses a MeanFlow objective as in Eq. (9), conditioned on the main feature $\boldsymbol{m} = \boldsymbol{m}_\theta(x_{t_i}, t_i)$; see Algorithm 2 for the pseudocode. Specifically, we parametrize the conditional inner flow map using an average velocity as

$$\boldsymbol{f}_\theta(\boldsymbol{y}_s, s, r; \boldsymbol{m}) \coloneqq \boldsymbol{y}_s + (s-r)\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r; \boldsymbol{m}). \tag{13}$$

Note that we do *not* detach the main backbone features during training as this would limit the flexibility needed in pretraining. Naively training the flow head to predict the average velocity $\boldsymbol{u}_\theta$ performs poorly. Our hypothesis is that the flow head's output should remain close to the pretrained teacher's output. Since the teacher predicts the velocity of the *outer* flow in Eq. (2), the flow head should instead predict $\mathbb{E}[\boldsymbol{x}_1 - \boldsymbol{x} \mid \boldsymbol{x}]$ to stay aligned with the teacher. From the inner velocity definition in Eq. (5), we obtain

$$\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, s; \boldsymbol{m}) \approx \mathbb{E}[\boldsymbol{y}_1 - (\boldsymbol{x}_1 - \boldsymbol{x}) | \boldsymbol{y}, \boldsymbol{x}_t]$$

Thus, we parametrize the average velocity as

$$\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r; \boldsymbol{m}) \coloneqq \boldsymbol{y}_1 - \mathrm{head}_\theta(\boldsymbol{y}_s, s, r; \boldsymbol{m}), \tag{14}$$

where head_θ is the head of our decoupled architecture (initialized from the teacher as outlined in Appendix A). With this parameterization, the head_θ output approximates the teacher velocity prediction in the limit of $r \to s$.

For improved performance and stability, we follow the original MeanFlow to (1) perform flow matching (more precisely, transition matching in our setting) for a part of the batch, (2) use classifier-free guidance (CFG) (adapting the conditional velocity $\boldsymbol{v}(\boldsymbol{y}_s, s)$), drop the text condition with a certain probability, and (3) use adaptive loss normalization. Since JVP computation in Eq. (10) requires custom implementations to be compatible with large-scale training for video generation (*e.g.*, flash attention [12], Fully Sharded Data Parallel (FSDP) [66], or context parallelism [23]), we use a finite-difference approximation of the JVP to make our algorithm agnostic of the underlying architectures and training techniques [47, 52]; see Appendix A for details.

Since we do not have direct access to the inner flow velocity, we use the conditional velocity $\boldsymbol{v}(\boldsymbol{y}_s, s) = \boldsymbol{y}_1 - \boldsymbol{y}$ in the objective (9). We note that for specific $\boldsymbol{y}$, one can also derive representations of the inner velocity in terms of the pretrained teacher velocity [20], which we leave for future work. Finally, we also observed that TM can provide competitive results as a pretraining strategy; see Section 4.3 for an ablation. In particular, TM pretraining can be understood as a special case of MeanFlow in Eq. (9) for $r = s$ when using the conditional velocity.

3.2. Stage 2: Distillation with flow head

After the TM-MF pretraining, we apply distribution distillation to align the student and teacher distributions. We significantly improve the baseline DMD2 for video models (see Appendix B) and leverage our optimized implementation for our TMD method.

DMD2-*v*. DMD2 was originally proposed for distilling image diffusion models. Thus, its design choices may not be optimal in the video domain. We identify three key factors that improve DMD2 for videos (termed DMD2-*v*), which serve as the default setting for our TMD training: (1) *GAN discriminator architecture*. We find that using Conv3D layers in the GAN discriminator outperforms other architectures, implying the importance of localized, spatio-

temporal features for the GAN loss; (2) *Knowledge distillation (KD) warm-up.* We find that KD warm-up improves the performance in one-step distillation, however, in multi-step generation, it tends to introduce coarse-grained artifacts that can be hardly fixed by the DMD2 training (see Figure 10 in Appendix). Therefore, we only apply KD warm-up in one-step distillation for DMD-v. (3) *Timestep shifting.* When we sample a timestep for the outer transition step or for adding noise to generated samples in the VSD loss, we find that applying a shifting function $t = \frac{\gamma t'}{(\gamma-1)t'+1}$ with $\gamma \geq 1$ to a uniformly sampled t' improves the performance and prevents mode collapse.

Flow head rollout. During distillation, we unroll the inner flow and treat the resulting architecture as a sample generator $\boldsymbol{g}_\theta(\boldsymbol{x}_{t_i}, t_i; \boldsymbol{y}_1)$ at each transition step t_i (see Figure 2b). Considering the flow head target $\boldsymbol{y} = \boldsymbol{x}_1 - \boldsymbol{x}$ defined in Eq. (3), the unrolled student output is given by

$$\boldsymbol{g}_\theta(\boldsymbol{x}_{t_i}, t_i; \boldsymbol{y}_1) := \boldsymbol{x}_1 - \textsc{InnerFlow}(\boldsymbol{m}_\theta(\boldsymbol{x}_{t_i}, t_i)) \quad (15)$$

where $\hat{\boldsymbol{y}}_0 \approx \textsc{InnerFlow}(\boldsymbol{m}_\theta(\boldsymbol{x}_{t_i}, t_i))$ denotes the final prediction of the flow head after N inner refinement steps, following Eq. (12).

Applying the VSD loss in Eq. (11) from DMD2-v to the unrolled student output naturally backpropagate gradients through all N inner flow steps. Since the flow head is lightweight, this remains efficient: for instance, if we choose 5 final DiT blocks from a 30-block DiT and unroll 2 steps, it adds less than 17% extra computation in updating the student network's parameters. In the terminology of DMD2, this can be viewed as "backward simulation" [58] of the inner flow, but without detaching the gradient computation. Distillation with flow head rollout effectively avoids the mismatch between training and inference, and thus improves the distillation performance.

Summary: We provide pseudocode for the student updates for both two stages in Algorithm 2 of Appendix A.

4. Experiments

4.1. Experiment setup

Implementation. We use Wan2.1 1.3B and 14B T2V-480p [49] as teacher video diffusion models and distill them to the same-sized student models with a decoupled architecture. All experiments are performed on the latent resolution of $[T, H, W] = [21, 60, 104]$, which gets decoded to 81 frames with pixel resolution 480×832. We use a dataset of 500k text and video pairs, where text prompts are sampled from the VidProM dataset [51] (and extended by Qwen-2.5 [55]) and videos are generated by the Wan2.1 14B T2V model. We defer more implementation details and hyperparameters to Appendix A.

Evaluation metrics. To evaluate our method and baselines, we use VBench [22] (where we report total score, quality score and semantic score) and a user preference study to assess visual quality and prompt adherence. We consider the *effective* number of function evaluations (NFE) to be the total number of DiT blocks used during generation divided by L, the number of blocks in the teacher architecture; for the baselines this corresponds to the number of steps M, and for our TMD models, this corresponds to

$$\text{Effective NFE} := M\big(1 + \tfrac{(N-1)H}{L}\big), \quad (16)$$

where N is the number of inner flow steps and H is the number of blocks in the flow head. We note that $L = 30$ for Wan2.1 1.3B and $L = 40$ for Wan2.1 14B.

Method	NFE	Overall score	Quality score	Semantic score
Diffusion models				
VideoCrafter2 1.4B [7]	50×2	80.44	82.20	73.42
CogVideoX-2B [57]	50×2	81.55	82.48	77.81
Seaweed-7B [43]	25×2	82.15	84.36	73.31
Wan2.1 1.3B [49]	50×2	84.26	85.30	80.09
Distilled models				
DOLLAR [13]‡	4	82.57	83.83	77.51
T2V-Turbo-v2 [28]*	4	82.34	83.93	75.97
rCM [69]	4	84.43	85.38	80.63
DMD2-v	4	84.60	86.03	79.87
APT [29]†	2	81.85	84.39	71.70
rCM [69]	2	84.09	84.90	**80.86**
DMD2-v	2	84.39	85.65	79.32
DMD2-v	3	84.48	<u>85.71</u>	79.58
TMD-N2H5 (Ours)	2.33	**84.68**	<u>85.71</u>	<u>80.55</u>
TMD-N4H5 (Ours)	3.00	<u>84.67</u>	**85.72**	80.47
APT [29]†	1	82.00	84.21	73.15
rCM [69]	1	82.65	83.60	78.82
DMD2-v	1	83.24	84.28	**79.10**
TMD-N2H5 (Ours)	1.17	**83.80**	**85.07**	78.69
TMD-N4H5 (Ours)	1.50	<u>83.79</u>	<u>85.03</u>	<u>78.83</u>

Table 1. VBench results of our method and baselines when distilling Wan2.1 1.3B into a few-step generator (*The teacher model is VideoCrafter2 1.4B [7]; †The teacher model is Seaweed-7B [43]; ‡The teacher model is a variant of CogVideoX [57]). DMD2-v denotes our improved version of DMD2 for video generation. "N2H5" (or "N4H5") means two (or four) denoising steps and five DiT blocks in the flow head. NFE in our method is represented by the effective forward pass of the whole network as in Eq. (16).

4.2. Comparison with existing methods

Our TMD method is built upon the improved version of DMD2 for video generation (termed DMD2-v). Here, we compare TMD with DMD2-v and existing baselines in distilling video diffusion models. We provide a visual comparison in Figure 3 and refer to Appendix B for further examples. In Table 1, we show the VBench results of distilling Wan2.1 1.3B (or video models with a similar size) into a few-step generator, where we group the distilled models based on the number of student denoising steps M. When $M = 2$, TMD-N2H5 with effective NFE $= 2.33$ (i.e., 2 denoising steps and 5 DiT blocks in flow head) obtains an



Figure 3. **Visual comparison.** We compare three frames (and zoomed-in regions of interest) of the outputs of TMD and DMD2-*v* on exemplary prompts for Wan2.1 1.3B (**left**) and Wan2.1 14B (**right**). TMD can improve visual quality at comparable cost to our DMD2-*v* baseline. Extended prompts can be found in Appendix A.

Method	NFE	Overall score	Quality score	Semantic score
Diffusion models				
Hunyuan Video 13B [25]	50×2	83.43	85.07	76.88
Wan2.1 14B [49]	50×2	86.22	86.67	84.44
Distilled models				
rCM [69]	4	84.92	85.43	82.88
DMD2-*v*	4	84.52	85.37	81.11
rCM [69]	2	**85.05**	85.57	**82.95**
DMD2-*v*	2	84.79	**85.78**	80.83
TMD-N4H5 (Ours)	2.75	84.92	85.54	82.42
rCM [69]	1	83.02	83.57	80.81
DMD2-*v*	1	83.69	84.46	80.61
TMD-N4H5 (Ours)	1.38	**84.24**	**84.89**	**81.65**

Table 2. VBench results of our method and baselines when distilling Wan2.1 14B into a few-step generator. DMD2-*v* denotes our improved version of DMD2 for video generation.

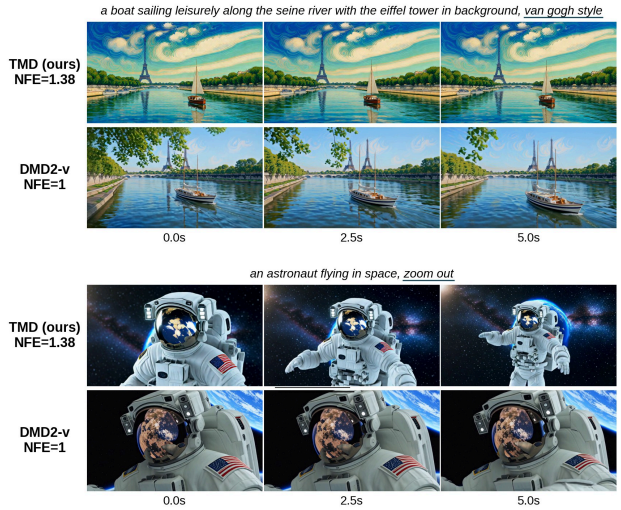


Figure 4. **Visual comparison.** We compare three frames of TMD and DMD2-*v* on exemplary prompts for Wan2.1 14B. TMD can improve prompt adherence at comparable cost to our DMD2-*v* baseline. Extended prompts can be found in Appendix A.

overall score 84.68 that outperforms all other distilled models, including the strongest baseline rCM with NFE $= 4$ (overall score 84.43). When $M = 1$, TMD-N2H5 with NFE $= 1.17$ also outperforms all other one-step distillation methods with an overall score 83.80, closing the gap from two-step distillation counterparts. With TMD, we can have more fine-grained control over the quality-efficiency tradeoff by allowing for *fractional* NFEs.

In Table 2, we show the VBench results of distilling Wan2.1 14B into a few-step generator. When $M = 2$, TMD-N4H5 with NFE $= 2.75$ does not outperform 2-step baselines (although it outperforms 4-step DMD-*v*). When $M = 1$, TMD-N4H5 with NFE $= 1.38$ significantly outperforms all other one-step distillation methods with an overall score of 84.24. It improves over one-step rCM by +1.22 while adding only minimal inference cost. In Figure 4, we show TMD improves prompt adherence, supporting the numerical improvement. Besides, TMD eliminates the need for the computationally expensive KD warm-up required by one-step DMD2-*v*.

User preference study. We perform a blinded *two-alternative forced choice* (2AFC) user preference study comparing TMD-N4H5 with DMD2-*v* when distilling Wan2.1 14B under two settings: (1) $M = 2$ (two-step generation) and (2) $M = 1$ (one-step generation). We randomly sample 60 challenging prompts from VBench [22] and generate 5 videos with different seeds for each prompt and model. For each prompt, raters are presented with side-by-side videos from the our model and the baseline (in ran-

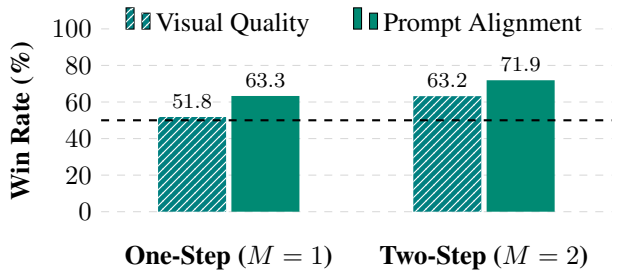


Figure 5. **User preference study results.** Comparison of TMD-N4H5 (ours) against DMD2-*v* under one-step ($M = 1$) and two-step ($M = 2$) distillation regimes. Values indicate the percentage of times users preferred our method over the baseline DMD2-*v* and the dashed line at 50% represents parity.

Discriminator head	Overall score	Quality score	Semantic score
Conv3D	**83.24**	**84.28**	**79.10**
Conv1D-2D	82.32	83.54	77.44
Attention	82.36	83.32	78.51
w/o GAN	81.63	82.70	77.35

Table 3. Impact of discriminator head design in DMD2-v for one-step distillation of Wan2.1 1.3B. We compare three heads: (1) *Conv3D*, jointly processing spatio-temporal features; (2) *Conv1D–2D*, separating temporal and spatial convolutions (e.g., [64]); and (3) *Attention*, flattening features into tokens processed by self-attention (with pooling downsampling).

KD warm-up	Overall score	Quality score	Semantic score
One-step w/ KD	**83.24**	84.28	**79.10**
One-step w/o KD	83.06	**84.72**	76.42
Two-step w/ KD	83.79	84.67	**80.28**
Two-step w/o KD	**84.39**	**85.65**	79.32

Table 4. Impact of the KD warm-up, where we distill Wan2.1 1.3B into a one-step or two-step student, respectively. For KD warm-up, we use teacher model to generate 10k noise-data pairs.

dom order and with random seed) and asked to perform independent pairwise comparisons for two separate criteria: *visual quality* and *prompt alignment*. Detailed instructions are provided in Figure 20.

As shown in Figure 5, users consistently preferred TMD over DMD2-*v* in both the one-step and two-step generation settings. The advantage is even more significant for prompt alignment, echoing the qualitative results in Figure 4 and underscoring the role of iterative flow-head refinement in largely improving prompt adherence.

4.3. Ablation studies

Three design choices in DMD2-*v*. Tables 3-5 show the impact of the discriminator head, KD warm-up and timestep shifting in DMD2-*v*, respectively. In Table 3, we observe

Timestep shifting	Overall score	Quality score	Semantic score
t_{dmd} w/ shift ($\gamma = 5$, one-step)	**83.24**	84.28	**79.10**
t_{dmd} w/o shift (one-step)*	83.22	**84.29**	78.94
t_{student} w/ shift ($\gamma = 10$, two-step)	**84.39**	**85.65**	**79.32**
t_{student} w/o shift (two-step)	83.44	84.96	77.33

Table 5. Impact of the timestep shifting for t_{dmd} that controls the noise level in the DMD loss and t_{student} that controls the denoising steps in the multi-step student, where we distill Wan2.1 1.3B into a one-step or two-step student, respectively. *Note that the setting "t_{dmd} w/o shift" leads to the severe mode collapse that VBench scores cannot capture (see Fig. 10 in Appendix).

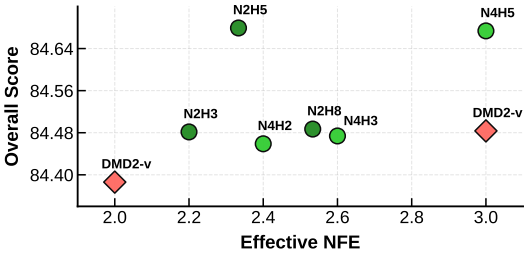


Figure 6. **Performance-efficiency tradeoff of TMD.** We compare the overall VBench score and effective NFE of TMD, when distilling Wan2.1 1.3B with $M = 2$ for different number of inner steps N and flow head layers H against 2- and 3-step DMD2-*v*. TMD can provide consistent performance gains for increasing NFE.

that adding the GAN loss improves the distillation performance, and *Conv3D* outperforms the other two discriminator head architectures. In Table 4, the overall score on VBench increases with KD warm-up in one-step DMD2 but decreases with KD warm-up in two-step DMD2. It implies that we better only apply KD warm-up in one-step generation. In Table 5, we observe that applying timestep shifting to t_{dmd} that controls the noise level in the DMD loss and t_{student} that controls the denoising steps in the multi-step student improves the distillation performance, respectively.

Quality-efficiency tradeoff. The number of inner steps N and flow head layers H control the computational cost of inner flows. We vary N and H to more comprehensively analyze TMD's performance-efficiency tradeoff. In Figure 6, we observe that the overall VBench score generally improves as the effective NFE increases. This justifies the fine-grained flexibility that our method offers in balancing generation speed and visual quality.

MeanFlow *vs.* flow matching. In transition matching pretraining, we replace the MeanFlow objective with the vanilla flow matching objective (TM) to highlight the impact of MeanFlow (TM-MF). Table 6 shows that TM-MF consistently achieves better distillation performance than TM, suggesting that TM-MF offers a superior initialization for the second-stage distillation training.

Setting	Pretraining	Overall score	Quality score	Semantic score
N2H5	TM-MF	**84.68**	**85.71**	**80.55**
	TM	84.61	85.64	80.46
N4H5	TM-MF	**84.67**	**85.72**	**80.47**
	TM	84.29	85.45	79.68

Table 6. Impact of the first stage pretraining method on the final performance when we distill Wan2.1 1.3B with $M = 2$.

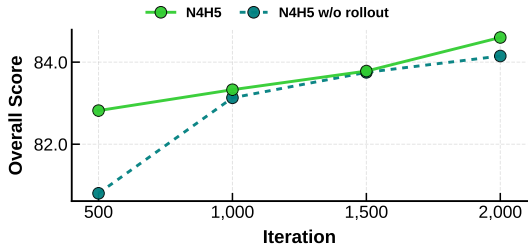


Figure 7. **Convergence and rollout ablation.** We compare the overall VBench score over iterations for the second-stage TMD training with and without flow head rollout. While TMD generally converges within a only few thousand iterations, we observe faster convergence and improved performance when using rollouts.

Flow head rollout in distillation. It is crucial to close the gap between training and inference, by allowing gradients from the distillation objective to backpropagate through the unrolled inner flow trajectory. Figure 7 shows that applying flow head rollout in distillation largely leads to faster training convergence and improved performance.

5. Related works

Accelerating diffusion models. Recent efforts to accelerate diffusion models for few-step generation generally follow two directions: Trajectory matching methods directly learn mappings along the underlying ODE trajectory [4, 8, 14, 16, 17, 26, 35, 39, 40, 46, 47, 68], while distribution matching methods train a student model to match the teacher's distribution with fewer steps [37, 41, 42, 54, 58, 59, 70]. In trajectory matching, Progressive Distillation [40] and Shortcut Models [14] gradually reduce sampling steps during training, while Consistency Models [16, 35, 46] directly map intermediate states to clean data. Subsequent works [4, 14, 17, 24, 39] extend this idea to learn mappings (or *flow maps*) between any pair of timesteps. They often face training instability when scaled to larger models or higher resolutions [9, 35, 69]. Distribution matching approaches, in contrast, align the student's few-step sampling distribution with the teacher's via distributional divergence [54, 58, 59]. This provides stable supervision at the distribution level and is less sensitive to the curvature of ODE trajectories, with adversarial objectives further improving sample fidelity.

As video generation grows in prominence along with its much higher sampling cost, recent works have extended these acceleration techniques to the video domain [13, 29, 64, 69]. Our method advances this line by introducing a decoupled architecture that more effectively leverages the teacher's learned semantic representations.

Efficient video generation. Beyond reducing sampling steps, other research aims to enhance the overall efficiency of video diffusion models. Autoregressive–diffusion hybrids achieve this by training causal models that generate frames sequentially instead of predicting the entire sequence at once [6, 11, 21, 30, 48, 60, 62]. This design significantly reduces attention complexity per forward pass through shorter temporal contexts. Meanwhile, system-level optimizations such as feature caching and efficient attention further reduce sampling cost by reusing intermediate activations [33, 38, 65] or by sparsifying or linearizing the attention operation [10, 53, 56]. These techniques are complementary to ours and can be combined with TMD for even greater video generation efficiency.

Decoupled backbone. Recent studies have demonstrated the advantages of decoupling diffusion backbones into a feature extractor (encoder) and a head (decoder) [27, 44, 50, 61, 67]. For example, aligning intermediate features with those from pretrained visual encoders improves training convergence and condition adherence [3, 61, 63]. Decoupled Diffusion Transformer (DDT) [50] further accelerates sampling by reusing extracted features across denoising steps, though such encoder sharing is not explicitly enforced during training. Transition Matching (TM) [44] explicitly models an inner flow loop for the head via flow matching [31]. Our work extends TM in two key aspects: (1) scaling it to large-scale video generation, and (2) distilling pretrained video diffusion models into few-step generators instead of training a multi-step TM model from scratch.

6. Conclusions

We introduced Transition Matching Distillation (TMD), a novel framework of addressing the significant inference latency of large-scale video diffusion models. The core of our method lies in a decoupled student architecture, which separates a main backbone for semantic feature extraction from a lightweight, recurrent flow head for iterative refinement. This design is coupled with a two-stage training strategy involving transition matching pretraining and distribution-based distillation. Our experiments on distilling Wan2.1 models showed that TMD offers fine-grained flexibility in balancing the trade-off between generation speed and video quality. TMD can outperform existing distillation techniques, achieving superior visual fidelity and prompt adherence at comparable or even reduced computational budgets. Future directions include (1) unifying the two training stages into a single-stage pipeline and (2) integrating TMD with system-level optimizations, such as efficient attention or feature caching, to further accelerate video generation.

References

[1] Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025. 1, 2

[2] Michael Samuel Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *The Eleventh International Conference on Learning Representations*, 2023. 2

[3] Aritra Bhowmik, Denis Korzhenkov, Cees GM Snoek, Amirhossein Habibian, and Mohsen Ghafoorian. MoAlign: Motion-centric representation alignment for video diffusion models. *arXiv preprint arXiv:2510.19022*, 2025. 8

[4] Nicholas M Boffi, Michael S Albergo, and Eric Vanden-Eijnden. Flow map matching. *arXiv preprint arXiv:2406.07507*, 2024. 3, 8

[5] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators, 2024. openai.com/research/video-generation-models-as-world-simulators. 1

[6] Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *Advances in Neural Information Processing Systems*, 37:24081–24125, 2024. 8

[7] Haoxin Chen, Yong Zhang, Xiaodong Cun, Menghan Xia, Xintao Wang, Chao Weng, and Ying Shan. Videocrafter2: Overcoming data limitations for high-quality video diffusion models, 2024. 5

[8] Hansheng Chen, Kai Zhang, Hao Tan, Leonidas Guibas, Gordon Wetzstein, and Sai Bi. pi-flow: Policy-based few-step generation via imitation distillation, 2025. 8

[9] Junsong Chen, Shuchen Xue, Yuyang Zhao, Jincheng Yu, Sayak Paul, Junyu Chen, Han Cai, Song Han, and Enze Xie. Sana-sprint: One-step diffusion with continuous-time consistency distillation. *arXiv preprint arXiv:2503.09641*, 2025. 8

[10] Junsong Chen, Yuyang Zhao, Jincheng Yu, Ruihang Chu, Junyu Chen, Shuai Yang, Xianbang Wang, Yicheng Pan, Daquan Zhou, Huan Ling, et al. Sana-video: Efficient video generation with block linear diffusion transformer. *arXiv preprint arXiv:2509.24695*, 2025. 8

[11] Justin Cui, Jie Wu, Ming Li, Tao Yang, Xiaojie Li, Rui Wang, Andrew Bai, Yuanhao Ban, and Cho-Jui Hsieh. Self-forcing++: Towards minute-scale high-quality video generation. *arXiv preprint arXiv:2510.02283*, 2025. 8

[12] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022. 4

[13] Zihan Ding, Chi Jin, Difan Liu, Haitian Zheng, Krishna Kumar Singh, Qiang Zhang, Yan Kang, Zhe Lin, and Yuchen Liu. DOLLAR: Few-step video generation via distillation and latent reward optimization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 17961–17971, 2025. 5, 8

[14] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. *arXiv preprint arXiv:2410.12557*, 2024. 8

[15] Yu Gao, Haoyuan Guo, Tuyen Hoang, Weilin Huang, Lu Jiang, Fangyuan Kong, Huixia Li, Jiashi Li, Liang Li, Xiaojie Li, et al. Seedance 1.0: Exploring the boundaries of video generation models. *arXiv preprint arXiv:2506.09113*, 2025. 1

[16] Zhengyang Geng, Ashwini Pokle, William Luo, Justin Lin, and J Zico Kolter. Consistency models made easy. *arXiv preprint arXiv:2406.14548*, 2024. 2, 8

[17] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025. 2, 3, 8, 12

[18] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020. 3

[19] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 1

[20] Peter Holderrieth, Uriel Singer, Tommi Jaakkola, Ricky TQ Chen, Yaron Lipman, and Brian Karrer. GLASS flows: Transition sampling for alignment of flow and diffusion models. *arXiv preprint arXiv:2509.25170*, 2025. 4

[21] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *arXiv preprint arXiv:2506.08009*, 2025. 8, 15

[22] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 2, 5, 6, 14

[23] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023. 4

[24] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ode trajectory of diffusion. *arXiv preprint arXiv:2310.02279*, 2023. 8

[25] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, et al. Hunyuanvideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024. 1, 2, 6

[26] Sangyun Lee, Yilun Xu, Tomas Geffner, Giulia Fanti, Karsten Kreis, Arash Vahdat, and Weili Nie. Truncated consistency models, 2024. 8

[27] Jiachen Lei, Keli Liu, Julius Berner, Haiming Yu, Hongkai Zheng, Jiahong Wu, and Xiangxiang Chu. Advancing end-to-end pixel space generative modeling via self-supervised pre-training. *arXiv preprint arXiv:2510.12586*, 2025. 4, 8

[28] Jiachen Li, Qian Long, Jian Zheng, Xiaofeng Gao, Robinson Piramuthu, Wenhu Chen, and William Yang Wang. T2v-turbo-v2: Enhancing video model post-training through data, reward, and conditional guidance design. In *The Thirteenth International Conference on Learning Representations*, 2025. 2, 5

[29] Shanchuan Lin, Xin Xia, Yuxi Ren, Ceyuan Yang, Xuefeng Xiao, and Lu Jiang. Diffusion adversarial post-training for one-step video generation. *arXiv preprint arXiv:2501.08316*, 2025. 5, 8

[30] Shanchuan Lin, Ceyuan Yang, Hao He, Jianwen Jiang, Yuxi Ren, Xin Xia, Yang Zhao, Xuefeng Xiao, and Lu Jiang. Autoregressive adversarial post-training for real-time interactive video generation. *arXiv preprint arXiv:2506.09350*, 2025. 8

[31] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. 8

[32] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. 2

[33] Haozhe Liu, Wentian Zhang, Jinheng Xie, Francesco Faccio, Mengmeng Xu, Tao Xiang, Mike Zheng Shou, Juan-Manuel Perez-Rua, and Jürgen Schmidhuber. Faster diffusion via temporal attention decomposition. *arXiv preprint arXiv:2404.02747*, 2024. 8

[34] Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2023. 2, 15

[35] Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. *arXiv preprint arXiv:2410.11081*, 2024. 2, 8

[36] Eric Luhman and Troy Luhman. Knowledge distillation in iterative generative models for improved sampling speed. *arXiv preprint arXiv:2101.02388*, 2021. 2

[37] Weijian Luo, Zemin Huang, Zhengyang Geng, J Zico Kolter, and Guo-jun Qi. One-step diffusion distillation through score implicit matching. *Advances in Neural Information Processing Systems*, 37:115377–115408, 2024. 8

[38] Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. *Advances in Neural Information Processing Systems*, 37:133282–133304, 2024. 8

[39] Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align your flow: Scaling continuous-time flow map distillation. *arXiv preprint arXiv:2506.14603*, 2025. 8

[40] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022. 2, 8

[41] Axel Sauer, Frederic Boesel, Tim Dockhorn, Andreas Blattmann, Patrick Esser, and Robin Rombach. Fast high-resolution image synthesis with latent adversarial diffusion distillation. In *SIGGRAPH Asia 2024 Conference Papers*, pages 1–11, 2024. 2, 8

[42] Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. In *European Conference on Computer Vision*, pages 87–103. Springer, 2024. 2, 8

[43] Team Seawead, Ceyuan Yang, Zhijie Lin, Yang Zhao, Shanchuan Lin, Zhibei Ma, Haoyuan Guo, Hao Chen, Lu Qi, Sen Wang, et al. Seaweed-7b: Cost-effective training of video generation foundation model. *arXiv preprint arXiv:2504.08685*, 2025. 5

[44] Neta Shaul, Uriel Singer, Itai Gat, and Yaron Lipman. Transition matching: Scalable and flexible generative modeling. *arXiv preprint arXiv:2506.23589*, 2025. 2, 4, 8

[45] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020. 1

[46] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning*, pages 32211–32252. PMLR, 2023. 2, 8

[47] Peng Sun, Yi Jiang, and Tao Lin. Unified continuous generative models. *arXiv preprint arXiv:2505.07447*, 2025. 3, 4, 8

[48] Hansi Teng, Hongyu Jia, Lei Sun, Lingzhi Li, Maolin Li, Mingqiu Tang, Shuai Han, Tianning Zhang, WQ Zhang, Weifeng Luo, et al. Magi-1: Autoregressive video generation at scale. *arXiv preprint arXiv:2505.13211*, 2025. 8

[49] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. 1, 2, 5, 6

[50] Shuai Wang, Zhi Tian, Weilin Huang, and Limin Wang. DDT: Decoupled diffusion transformer. *arXiv preprint arXiv:2504.05741*, 2025. 4, 8

[51] Wenhao Wang and Yi Yang. Vidprom: A million-scale real prompt-gallery dataset for text-to-video diffusion models. *Advances in Neural Information Processing Systems*, 37: 65618–65642, 2024. 5

[52] Zidong Wang, Yiyuan Zhang, Xiaoyu Yue, Xiangyu Yue, Yangguang Li, Wanli Ouyang, and Lei Bai. Transition models: Rethinking the generative learning objective. *arXiv preprint arXiv:2509.04394*, 2025. 3, 4

[53] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, et al. Sparse videogen: Accelerating video diffusion transformers with spatial-temporal sparsity. *arXiv preprint arXiv:2502.01776*, 2025. 8

[54] Yilun Xu, Weili Nie, and Arash Vahdat. One-step diffusion models with f-divergence distribution matching. *arXiv preprint arXiv:2502.15681*, 2025. 8

[55] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. 5, 15

[56] Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Kelly Peng, et al. Sparse videogen2: Accelerate video generation with sparse attention via semantic-aware permutation. *arXiv preprint arXiv:2505.18875*, 2025. 8

[57] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. In *The Thirteenth International Conference on Learning Representations*, 2025. 1, 5

[58] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and William T Freeman. Improved distribution matching distillation for fast image synthesis. In *NeurIPS*, 2024. 2, 3, 5, 8, 14

[59] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6613–6623, 2024. 2, 3, 8

[60] Tianwei Yin, Qiang Zhang, Richard Zhang, William T Freeman, Fredo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast autoregressive video diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 22963–22974, 2025. 8

[61] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion transformers is easier than you think. *arXiv preprint arXiv:2410.06940*, 2024. 4, 8

[62] Lvmin Zhang and Maneesh Agrawala. Packing input frame context in next-frame prediction models for video generation. *arXiv preprint arXiv:2504.12626*, 2025. 8

[63] Xiangdong Zhang, Jiaqi Liao, Shaofeng Zhang, Fanqing Meng, Xiangpeng Wan, Junchi Yan, and Yu Cheng. VideoREPA: Learning physics for video generation through relational alignment with foundation models. *arXiv preprint arXiv:2505.23656*, 2025. 8

[64] Zhixing Zhang, Yanyu Li, Yushu Wu, yanwu xu, Anil Kag, Ivan Skorokhodov, Willi Menapace, Aliaksandr Siarohin, Junli Cao, Dimitris N. Metaxas, Sergey Tulyakov, and Jian Ren. SF-v: Single forward video generation model. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. 2, 7, 8

[65] Xuanlei Zhao, Xiaolong Jin, Kai Wang, and Yang You. Real-time video generation with pyramid attention broadcast. *arXiv preprint arXiv:2408.12588*, 2024. 8

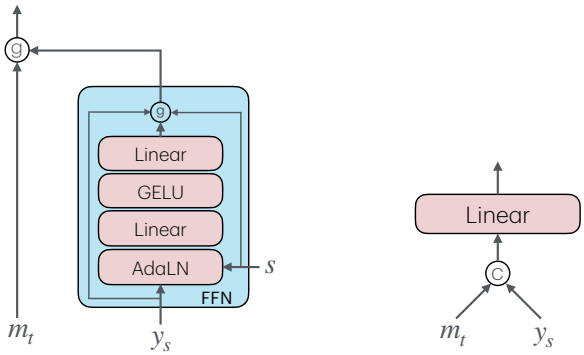
[66] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023. 4

[67] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion transformers with representation autoencoders. *arXiv preprint arXiv:2510.11690*, 2025. 4, 8

[68] Hongkai Zheng, Weili Nie, Arash Vahdat, Kamyar Azizzadenesheli, and Anima Anandkumar. Fast sampling of diffusion models via operator learning. In *International conference on machine learning*, pages 42390–42402. PMLR, 2023. 8

[69] Kaiwen Zheng, Yuji Wang, Qianli Ma, Huayu Chen, Jintao Zhang, Yogesh Balaji, Jianfei Chen, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Large scale diffusion distillation via score-regularized continuous-time consistency. *arXiv preprint arXiv: 2510.08431*, 2025. 2, 5, 6, 8, 17, 18

[70] Mingyuan Zhou, Huangjie Zheng, Zhendong Wang, Mingzhang Yin, and Hai Huang. Score identity distillation: Exponentially fast distillation of pretrained diffusion models for one-step generation. In *Forty-first International Conference on Machine Learning*, 2024. 2, 8



(a) Fusion type: "gated" (b) Fusion type: "concat"

Figure 8. Visualization of our mechanisms to fuse the main backbone's features $\boldsymbol{m}(\boldsymbol{x}, t)$ with the flow head input $\boldsymbol{y}_s$: (a) "gated" fusion, where the flow head input $\boldsymbol{y}_s$ is passed to an FFN block (conditioned on the flow head timestep s) and then added to the main feature $\boldsymbol{m}_t$ via a gated operator; and (b) "concat" fusion, where the the flow head input $\boldsymbol{y}_s$ and the main feature $\boldsymbol{m}_t$ are concatenated in the channel dimension and then passed to the linear projection layer.

Algorithm 1 TMD inference

$\boldsymbol{x} \sim \mathcal{N}(0, I)$
for $i = M$ **to** 1 **do**
  $\boldsymbol{m} = \boldsymbol{m}_\theta(\boldsymbol{x}, t_i)$ ▷ Main backbone
  $\boldsymbol{x} = \boldsymbol{x} - (t_i - t_{i-1})\textsc{InnerFlow}(\boldsymbol{m})$ ▷ See Eq. (4)
return $\boldsymbol{x}$ ▷ Generated data

def $\textsc{InnerFlow}(\boldsymbol{m})$
  $\boldsymbol{y} \sim \mathcal{N}(0, I)$
  for $j = N$ **to** 1 **do**
    $\boldsymbol{y} = \boldsymbol{f}_\theta(\boldsymbol{y}, s_j, s_{j-1}; \boldsymbol{m})$ ▷ Flow head
  return $\boldsymbol{y}$

A. Implementation details

A.1. Flow head conditioning

Time conditioning. For the flow head, we re-use the default time embedding to embed s and instantiate an additional zero-initialized time embedding that embeds the difference $s - r$ between two time steps of the flow map (see Section 2). The two embeddings are summed and used as the conditioning input to the adaptive normalization layers in the DiT blocks of the flow head.

Inner flow fusion. To fuse the main backbone's features $\boldsymbol{m}(\boldsymbol{x}, t)$ with the flow head input $\boldsymbol{y}_s$, we first process $\boldsymbol{y}_s$ with the patch embedding layer of the main backbone. Then we process the patches using a randomly initialized AdaLN-style block (analogous to the ones used in the teacher model, *i.e.*, consisting of adaptive layer normalization, an MLP, and a gated residual connection) conditioned on the embedding of s and re-using the global scale and shift values. Finally, we fuse this processed flow head input with the main backbone's features using a gated interpolation, controlled by a global gate derived from a sigmoid-activated learnable parameter. This makes the flow head's contribution both time-aware and dynamically gated at the feature level before the final global fusion. We visualize the fusion mechanism in Figure 8 and provide an ablation w.r.t. other conditioning types in Section B.3.

A.2. TM-MF

Pretraining recipe. To better align TM-MF pretraining with the multi-step inference strategy used in our distillation setup, we modify the sampling scheme of (s, r) compared to the continuous formulation in [17]. First, analogous to t_{student} in the outer flow of DMD2-*v*, we also use a shifting function with $\gamma = 10$ for s_{student}, i.e., the values defining the time grid[2] $0 = s_0 < s_1 < \cdots < s_N = 1$. Secondly, analogous to sampling the timestep t_{dmd} in DMD2-*v*, we also sample s_{mf} during TM-MF training uniformly and shift it (using $\gamma = 3$). Then, we pick $r = s_k$ where $k := \max\{j : s_j \leq s_{\text{mf}}\}$. Empirically, this discrete sampling scheme is not only tailored to our Stage 2 distillation but also substantially stabilizes TM-MF pretraining.

Our remaining design choices largely follow Geng et al. [17]. We set $r = s$ for 75% of training batches to stabilize optimization under the flow-matching loss, apply condition dropout (using the negative prompt in Table 8) during training, and construct the velocity field using classifier-free guidance. We observed that mixing conditional and unconditional network-predicted velocities (Eq. (18) in [17]) offers no clear benefit in our setting, so we adopt the standard classifier-free guidance formulation throughout. Moreover, we also use an adaptive loss normalization, such that our final loss derived from (9) is given by

$$\mathbb{E}_{s,r,\boldsymbol{y}_s}\left[\frac{\|\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r) - \hat{\boldsymbol{u}}\|^2}{\text{sg}\left(\|\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r) - \hat{\boldsymbol{u}}\|^2\right) + c}\right],$$

where we choose $c = d$ for Wan2.1 1.3B and $c = \frac{d}{10^5}$ for Wan2.1 14B (where d is the dimension of $\boldsymbol{y}_s$).

Finite difference approximation. Computing the JVP term in the MeanFlow objective, namely $\frac{\mathrm{d}}{\mathrm{d}s}\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r)$, using forward-mode automatic differentiation in PyTorch is currently incompatible with system optimizations, such as flash attention and FSDP; however, these optimizations are crucial to avoid out-of-memory issues due to the long video sequence length. To address this and make our method agnostic of different training techniques, we approximate the JVP using a *central difference scheme*. For a step size δ, we

[2]Note that, in practice, we set $t_M = s_N = 0.999$ instead of 1 to align with the pretraining of Wan.

Hyperparameter	Wan2.1 1.3B (T2V)	Wan2.1 14B (T2V)
General Settings		
Resolution	480×832 (480P)	
Frame count	81 Frames (5s)	
Latent dimension $(T \times H \times W)$	$21 \times 60 \times 104$	
Dataset size	500k (479k after filtering)	
Dataset prompts	VidProm extended by QWEN2.5-7B	
Dataset videos	Generated by Wan2.1 14B (T2V)	
Optimizer (weight decay, betas, epsilon)	AdamW $(0.01, (0.9, 0.99), 10^{-8})$	
Global batch-size	64	
Timestep shift γ **for** t_{student} (see Section 3.2)	10	
Multi-step sampling (see Section A.3)	deterministic	
Precision	BF16 (time t in FP64)	
GPU type	NVIDIA A100	NVIDIA H100
Parallelism Strategy	DDP	FSDP
Baselines		
KD		
Number of trajectories	10k	
CFG scale	5	
Skip-layer	10	
Learning rate	7e-5	
Max. iterations	10k	5k
DMD2-*v*		
Fake score architecture	same as teacher	
Discriminator architecture (see Section 4.3)	Conv3D	
Discriminator parameters	68M	172M
CFG scale	5	
Layer for discriminator features (see Section A.3)	$(15, 22, 29)$	$(19, 29, 39)$
Discriminator loss weight λ (see Algorithm 2)	0.03	
Learning rates (student, discriminator, fake score)	$(10^{-5}, 10^{-5}, 10^{-5})$	
Timestep shift γ **for** t_{dmd} (see Section 3.2)	5	
Min./max. for t_{dmd} **and** s_{mf}	$(0.001, 0.999)$	
Student update frequency	every 5-th iteration	
Max. iterations	4k $(M \geq 2)$, 12k $(M = 1)$	1k $(M \geq 2)$, 5k $(M = 1)$
TMD		
Stage 1: TM-MF		
CFG scale	3	
Flow head fuse type (see Section A.1)	gated	
Timestep shift γ **for** s_{student} (see Section A.2)	10	
Timestep shift γ **for** s_{mf} (see Section A.2)	3	
JVP finite-difference δ (see Section A.2)	$5 \cdot 10^{-3}$	
Condition dropout probability (see Section A.2)	0.1	
Loss normalization constant c (see Section A.2)	d	$10^{-5}d$
Learning rate	$3 \cdot 10^{-5}$	$1 \cdot 10^{-5}$
Max. iterations	3k	3k
Stage 2: DMD2-*v* with flow head		
All hyperparameters as in DMD2-*v* above		

Table 7. Default hyperparameters if not specified otherwise in the experiments.

Algorithm 2 TMD student update step (simplified)

Given $\boldsymbol{x} \sim p_{\text{data}},\ \boldsymbol{x}_1 \sim \mathcal{N}(0, I),\ t_i \sim \text{Unif}(\{t_1, \ldots, t_M\})$
$\boldsymbol{x}_{t_i} = (1 - t_i)\boldsymbol{x} + t_i \boldsymbol{x}_1$ ▷ See Eq. (1)
$\boldsymbol{m} = \boldsymbol{m}_\theta(\boldsymbol{x}_{t_i}, t_i)$ ▷ Main backbone
if stage_one **then**
  $\boldsymbol{y} = \boldsymbol{x}_1 - \boldsymbol{x}$ ▷ See Eq. (3)
  $\boldsymbol{y}_1 \sim \mathcal{N}(0, I),\ (s, r) \sim p_{s,r}$
  $\boldsymbol{y}_s = (1 - s)\boldsymbol{y} + s\boldsymbol{y}_1$ ▷ See Eq. (5)
  $\boldsymbol{u} = \boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r; \boldsymbol{m})$ ▷ Avg. velocity
  $\boldsymbol{v} = \boldsymbol{y}_1 - \boldsymbol{y}$ ▷ Conditional velocity
  $\mathcal{L} = \text{MeanFlow}(\boldsymbol{u}, \boldsymbol{v}, s, r)$ ▷ See Eq. (9)
else
  $\hat{\boldsymbol{x}} = \boldsymbol{x}_1 - \textsc{InnerFlow}(\boldsymbol{m})$ ▷ See Eq. (15) & Algorithm 1
  $\mathcal{L} = \text{VSD}(\hat{\boldsymbol{x}}) + \lambda \cdot \text{Discriminator}(\hat{\boldsymbol{x}})$ ▷ See Eq. (11)
$\theta = \text{step}(\theta, \nabla_\theta \mathcal{L})$ ▷ Gradient step

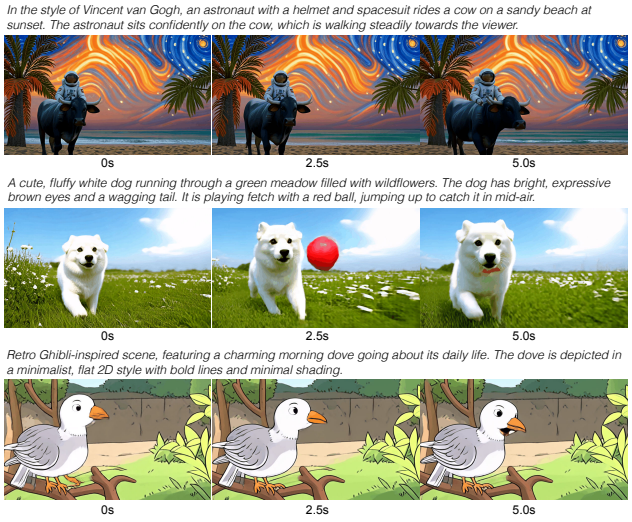


Figure 9. **Mode collapse without time-shifting.** We show videos generated by the one-step student distilled from DMD2 in the setting "t_{dmd} w/o shift" (see Table 5), where the other hyperparameters use the default values. We can see that all generated videos have the main characters consistently appear on the left side of the pixel space, which is a sign of the severe mode collapse happening during distillation.

have

$$\frac{\mathrm{d}}{\mathrm{d}s}\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r) \approx \frac{\boldsymbol{u}_\theta(\boldsymbol{y}_{s+\delta}, s+\delta, r) - \boldsymbol{u}_\theta(\boldsymbol{y}_{s-\delta}, s-\delta, r)}{2\delta}$$

with $\boldsymbol{y}_{s\pm\delta} = \boldsymbol{y}_s \pm \delta \boldsymbol{v}(\boldsymbol{y}_s, s)$, where we use the conditional velocity for $\boldsymbol{v}$. At the boundaries, we fall back to using one-sided finite-difference approximations. We empirically found that setting $\delta = 0.005$ yields a satisfactory estimation to the JVP term and fix it throughout our experiments.

A.3. DMD2

Initialization. Since we consider video diffusion models that are trained to approximate the instantaneous velocity $\boldsymbol{v}$ in (2), we parametrize the student network as

$$\boldsymbol{g}_\theta^{\text{student}}(\boldsymbol{x}_t, t) = \boldsymbol{x}_t - t\boldsymbol{v}_\theta(\boldsymbol{x}_t, t) \tag{17}$$

for our DMD2-v baseline, which initially approximates the conditional expectation $\mathbb{E}[\boldsymbol{x}|\boldsymbol{x}_t]$.

Multi-step distillation and inference. Compared to the original multi-step DMD2 [58], we did not use backward simulation for the outer loop for multi-step distillation and efficiently sample $\boldsymbol{x}_{t_i}$ using real data.

Moreover, during inference, we use deterministic sampling from the conditional flow, *i.e.*,

$$\boldsymbol{x}_{t_{i+1}} = \left(1 - \frac{t_{i+1}}{t_i}\right)\boldsymbol{x}_{t_i} + \frac{t_{i+1}}{t_i}\boldsymbol{g}^{\text{student}}(\boldsymbol{x}_{t_i}, t_i), \tag{18}$$

which samples from the correct distribution as long as $\boldsymbol{x} \approx \text{Student}(\boldsymbol{x}_{t_i}, t_i)$. Compared to the standard resampling scheme in DMD2, *i.e.*,

$$\boldsymbol{x}_{t_{i+1}} = (1 - t_{i+1})\,\boldsymbol{g}^{\text{student}}(\boldsymbol{x}_{t_i}, t_i) + t_{i+1}\boldsymbol{x}_1, \tag{19}$$

the noise $\boldsymbol{x}_1$ of the conditional flow is inferred from $\boldsymbol{g}^{\text{student}}(\boldsymbol{x}_{t_i}, t_i)$ and $\boldsymbol{x}_{t_i}$ in Eq. (18) and sampled independently in Eq. (19).

For TMD, the multi-step inference is given in Algorithm 1. In particular, the outer transitions are also deterministic, but additional independent noise $\boldsymbol{y}_1 \sim \mathcal{N}(0, I)$ is used for the inner flow.

Teacher, fake score, and discriminator. While it would be possible to distill Wan2.1 1.3B using the 14B model as teacher in DMD2, we use the 1.3B model for fair comparisons. Moreover, we use (unpatchified) teacher features at different layers as inputs to the discriminator, which we found more stable than using fake score features. The discriminator is trained using an average minimax log-likelihood objective over separate heads for each teacher feature, where the teacher is evaluated at noisy inputs $(\hat{\boldsymbol{x}}_t, t)$ (fake data as defined in Section 2) or $(\boldsymbol{x}_t, t)$ (real data). The generator loss is given as the average (non-saturating) negative log-likelihood. We initialize the fake score $\boldsymbol{g}^{\text{fake}}$ using the teacher parameters, parametrize it analogous to (17), and use denoising score matching to train it on noisy fake data $(\hat{\boldsymbol{x}}_t, t)$. Both the discriminator and fake score are trained for several iterations in between student updates. Finally, we can write our VSD objective in (11) as

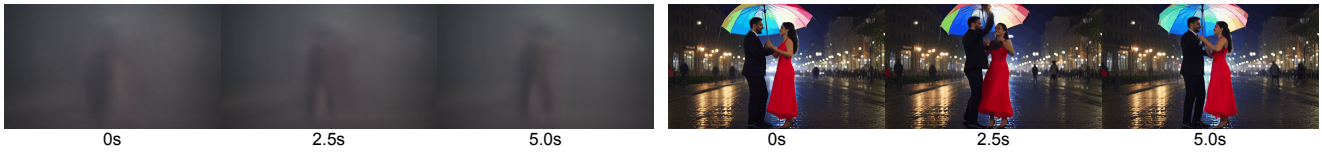
$$\mathbb{E}_{t_i, \boldsymbol{x}_{t_i}, t, \hat{\boldsymbol{x}}_t}\left[\text{sg}\left(\frac{\boldsymbol{g}^{\text{fake}}(\hat{\boldsymbol{x}}_t, t) - \boldsymbol{g}^{\text{teacher}}(\hat{\boldsymbol{x}}_t, t)}{\|\boldsymbol{g}^{\text{fake}}(\hat{\boldsymbol{x}}_t, t) - \boldsymbol{g}^{\text{teacher}}(\hat{\boldsymbol{x}}_t, t)\|_1}\right)^T \hat{\boldsymbol{x}}\right],$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operation, and the teacher is also parametrized analogous to Eq. (17), including CFG with the negative prompt given in Table 8.

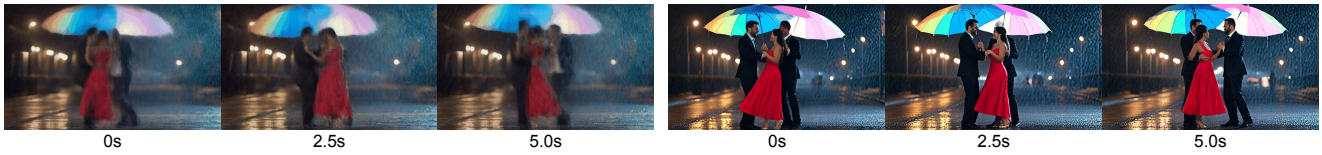
A.4. VBench evaluation

We follow the official evaluation protocol in VBench [22] to test our method. During video sampling, we use standard VBench prompt lists to ensure fair comparisons among different methods. In particular, we rewrite the test prompts (*i.e.*, prompt augmentation) using QWEN/QWEN2.5-7B-

A romantic scene featuring a happy couple dancing salsa in the rain under a large colorful umbrella. They are dressed in elegant evening attire, with the man in a black suit and the woman in a flowing red dress. Both have joyful expressions as they move gracefully to the music, their feet barely touching the wet pavement. Raindrops fall softly around them, creating a misty atmosphere.



(a) Two-step DMD2 w/o KD warm-up (*Left*: Iteration 0, *Right*: Iteration 1000)



(b) Two-step DMD2 w/ KD warm-up (*Left*: Iteration 0, *Right*: Iteration 1000)

Figure 10. **Effect of KD initialization.** We compare the two-step DMD2 results of distilling Wan2.1 1.3B in two settings: (a) with and (b) without KD warm-up, where (left) "Iteration 0" means videos generated in the beginning of DMD2 training and (Right) "Iteration 1000" means videos generated after training DMD2 for 1k iterations. From left to right, we show the first, middle and the last frames in each video. We can see that the KD warm-up initially can generate better videos, but it also introduces coarse-grained artifacts. For instance, it generates an extra man besides a couple specified in the prompt. After training for 1k iterations, two-step DMD2 cannot remove these artifacts, leading to the worse generation quality than two-step DMD2 without KD warm-up.

INSTRUCT [55], following the prior work [21]. Similarly, almost all the baselines, including Wan2.1 base models, also enhance VBench prompts, making them longer and more descriptive without altering their original meaning. We then calculate scores for 16 Text-to-Video (T2V) evaluation dimensions, respectively, and summarize them into quality score, semantic score and overall score.

In figures from the main text, we only show the short prompts due to the space limit. In Table 8, we show the corresponding extended prompts in Figures 1-4 that are actually passed to the model.

A.5. Hyperparameters

We provide an overview of our default hyperparameters in Table 7.

B. Additional experiments

B.1. Quality-efficiency tradeoff

Extending the experiments in Section 4.3, Figure 11 shows TMD's performance-efficiency tradeoff when varying both the number of outer and inner steps M and N and the flow head layers H. It shows a general trend of achieving better performance with a larger effective NFE and TMD offers a more fine-grained control over this performance-efficiency tradeoff than DMD2-*v*.

B.2. Curvature of Wan trajectories

Inspired by Liu et al. [34], we define the curvature of Wan's sampling trajectory at time t as

$$C(\boldsymbol{v}, t) = \|\boldsymbol{v}(\boldsymbol{x}_t, t) - (\boldsymbol{x} - \boldsymbol{x}_0)\|^2.$$

Since the model is evaluated on a fixed time grid during sampling, we adopt a discretized version of the curvature in practice, *i.e.*,

$$C(\boldsymbol{v}, t_i) = \left\| \frac{\tilde{\boldsymbol{x}}_{t_i} - \tilde{\boldsymbol{x}}_{t_{i-1}}}{t_i - t_{i-1}} - (\boldsymbol{x}_1 - \boldsymbol{x}) \right\|^2,$$

where t_i denotes the timesteps from Wan's default 50-step sampling schedule and $\tilde{\boldsymbol{x}}_{t_i}$ represents the corresponding intermediate samples along the trajectory.

As shown in Figure 12, the sampling trajectories of the Wan model exhibit extremely large curvature near $t = 1$ (*i.e.*, the high noise regime), making it difficult for trajectory matching methods to learn the mapping along the ODE path. This also motivates our choice of larger γ in the time-shifting function for r_i in TM-MF (see Section A.2) and t_{student} in DMD2-*v* (see Section 3.2).

B.3. Flow head conditioning

Our main experiments use the gating mechanism explained in Section A.1 to condition the flow head on the main backbone's features $\boldsymbol{m} = \boldsymbol{m}(\boldsymbol{x}, t)$. Another version is to concatenate $\boldsymbol{m}$ and the patch embeddings of $\boldsymbol{y}_s$ along the hidden dimension and then project to the original hidden size using a linear layer. To minimize the impact on the pretrained model, we initialized the weights of the linear layer as identity for the coordinates corresponding to $\boldsymbol{m}$ and as a Gaussian with small standard deviation (0.01) for the remaining coordinates. In Table 9, we see that TMD can achieve strong performance across fusion types. While convergence was more stable with the gating mechanism used for our main experiments, concatenation is a strong alternative in terms of final performance.

Figure 1	
Short prompt	*A fat rabbit wearing a purple robe walking through a fantasy landscape.*
Long prompt	*A plump, fluffy rabbit donning a voluminous purple robe walks gracefully through a vibrant fantasy landscape. The rabbit has large, expressive eyes and a gentle, curious expression. Its fur is soft and thick, and the robe drapes elegantly over its body. The landscape features rolling hills covered in lush green grass, colorful wildflowers, and towering magical trees with shimmering leaves. In the distance, there are sparkling waterfalls and mystical castles. The scene is bathed in warm, golden sunlight. Medium shot, focusing on the rabbit's walk through the picturesque environment.*
Short prompt	*A person drinking coffee in a cafe.*
Long prompt	*A cozy, warm café setting with soft ambient lighting and wooden furnishings. A young adult, casually dressed in a sweater and jeans, sits at a small round table. They hold a steaming cup of coffee in their hand, taking a sip while looking pensively out the window. The café is moderately busy with other patrons engaged in conversations. The background showcases various coffee drinks and pastries displayed on a counter. The person's expression is relaxed and content. Medium shot focusing on the person's face and the coffee cup, capturing the intimate atmosphere of the café.*
Figure 3	
Short prompt	*A person is laughing.*
Long prompt	*A joyful person is laughing heartily, with a broad smile and crinkled eyes, conveying pure happiness. They are standing upright with arms spread wide, as if embracing the world around them. The scene is set outdoors in a sunny park, surrounded by lush greenery and blooming flowers. The background includes a clear blue sky with fluffy clouds, adding to the cheerful atmosphere. Medium shot capturing the full body of the person, focusing on their animated facial expressions and gestures.*
Short prompt	*A steam train moving on a mountainside.*
Long prompt	*A vintage steam locomotive chugging along a winding track on a mountainous terrain. The train is covered in soot, with steam billowing from its smokestack as it navigates the rugged landscape. The surrounding mountains are steep and lush, with patches of snow visible at higher elevations. The train cars sway gently as they follow the curving tracks, and the scenery outside the windows shows dense forests and rocky cliffs. The camera follows the train from a medium distance, capturing the train's movement and the dramatic backdrop.*
Figure 4	
Short prompt	*A boat sailing leisurely along the Seine River with the Eiffel Tower in background.*
Long prompt	*A serene, picturesque scene of a small wooden boat gently gliding along the Seine River in Paris, France. The boat is rowed leisurely by a middle-aged man in a casual striped shirt and khaki pants, who rows smoothly with rhythmic strokes. The Eiffel Tower stands majestically in the background, partially visible through the misty morning air. The riverbank is lined with lush green trees and quaint buildings, reflecting off the calm waters. The overall atmosphere is peaceful and tranquil, capturing the essence of a lazy summer day. Wide shot, static camera.*
Short prompt	*An astronaut flying in space, zoom out.*
Long prompt	*Astronaut floating in space with a helmet visor reflecting Earth below. The astronaut, wearing a full spacesuit with the American flag on the shoulder, is performing a spacewalk, arms extended as if in motion. The background shows the vastness of space with stars twinkling and Earth in the distance. The scene begins with a close-up of the astronaut and gradually zooms out to reveal the enormity of space surrounding them. Wide shot, showcasing the astronaut against the backdrop of the universe.*
Negative prompt	
Bright tones, overexposed, static, blurred details, subtitles, style, works, paintings, images, static, overall gray, worst quality, low quality, JPEG compression residue, ugly, incomplete, extra fingers, poorly drawn hands, poorly drawn faces, deformed, disfigured, misshapen limbs, fused fingers, still picture, messy background, three legs, many people in the background, walking backwards	

Table 8. Extended prompts used in the figures from the main text and negative prompt used for CFG (in teacher sampling, KD, and DMD2-*v*) and condition dropout (in TM-MF), taken from the official Wan repository.

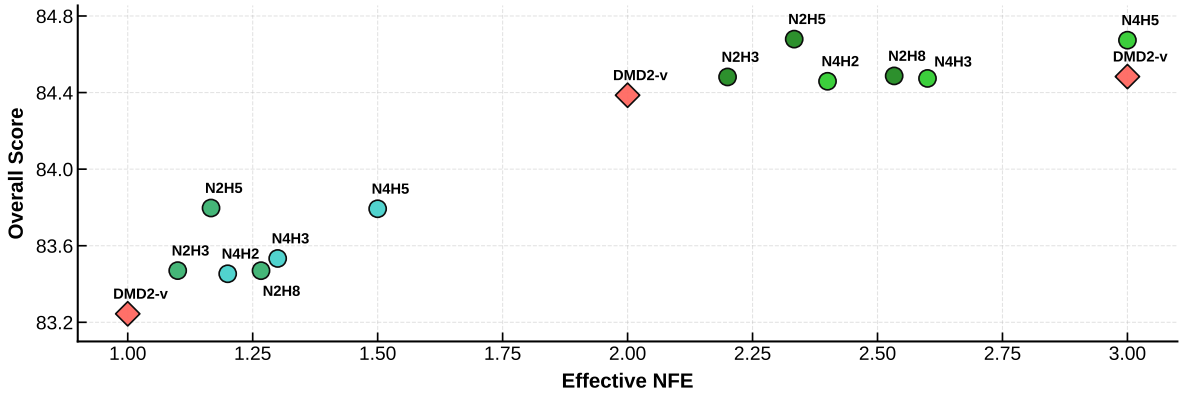


Figure 11. **Performance-efficiency tradeoff of TMD.** Extension of Figure 6 to include the $M = 1$ settings.

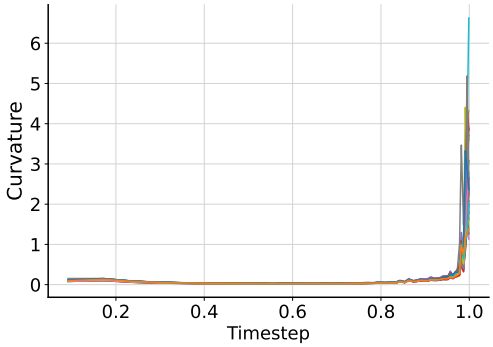


Figure 12. **Curvature of Wan trajectories.** Curvature of different sampling trajectories for the Wan2.1 1.3B model. Large trajectory curvatures are observed near $t = 1$ (*i.e.*, the high noise regime).

Setting	Fusion type	Overall score	Quality score	Semantic score
N2H5	Gated (see Section A.1)	84.68	85.71	80.55
	Concat	**84.76**	**85.77**	**80.71**
N4H5	Gated (see Section A.1)	**84.67**	85.72	**80.47**
	Concat	84.66	**85.79**	80.14

Table 9. Impact of the fusion type on the final performance when distilling Wan2.1 1.3B with $M = 2$.

B.4. Inner flow targets

In principle, the auxiliary latent variable $\boldsymbol{y}$ in the inner flow can be chosen arbitrarily as long as $\boldsymbol{x}_{t_{i-1}}$ is easy to sample given $\boldsymbol{y}$ and $\boldsymbol{x}_{t_i}$. While we chose the DTM formulation $\boldsymbol{y} := \boldsymbol{x}_1 - \boldsymbol{x}$ in (3) for our main experiments, we also experimented with other versions. While other formulations did not improve the overall performance, they could still achieve competitive performance. For instance, Table 10 provides results for the simple choice $\boldsymbol{y} := \boldsymbol{x}$. Similar to (14) and (17), we parametrize the average velocity as

$$\boldsymbol{u}_\theta(\boldsymbol{y}_s, s, r; \boldsymbol{m}) := \frac{\boldsymbol{y}_s - (\boldsymbol{x}_{t_i} - t_i \mathrm{head}_\theta(\boldsymbol{y}_s, s, r; \boldsymbol{m}))}{s}.$$

where $\boldsymbol{m} = \boldsymbol{m}_\theta(x_{t_i}, t_i)$ denote the main features.

B.5. Impact of recurrence in flow head

To evaluate the effect of recurrence in the flow head, we manually restrict the inner flow to a single step at inference (N1H5) and compare the results with the standard recurrent setting (N4H5). As shown in Figure 13, when distilling Wan2.1 1.3B into a two-step generator, the non-recurrent variant produces noticeably lower-quality videos, exhibiting stronger artifacts and blurriness. This highlights the importance of iterative refinement within the flow head for high-fidelity generation.

Setting	Target $\boldsymbol{y}$	Overall score	Quality score	Semantic score
N2H5	$\boldsymbol{x}_1 - \boldsymbol{x}$ (DTM)	**84.68**	**85.71**	**80.55**
	$\boldsymbol{x}$	84.18	85.15	80.33
N4H5	$\boldsymbol{x}_1 - \boldsymbol{x}$ (DTM)	**84.67**	**85.72**	80.47
	$\boldsymbol{x}$	84.44	85.36	**80.77**

Table 10. Impact of the inner flow target when distilling Wan2.1 1.3B with $M = 2$.

B.6. DMD2 comparisons

In Section 3.2, we observed that KD pretaining is only helpful for one-step distillation (see Table 4 and Figure 9) and that timestep shifting of t_{dmd} and t_{student} is crucial for good performance for one- and two-step DMD2-*v* (see Table 5 and Figure 10). In this section, we provide additional ablations on our DMD2-*v* baseline when using ≥ 3 steps. While the timestep shifting for t_{dmd} and t_{student} remain important, we observed that slightly lower shift values for t_{student} can attain better scores when using more steps. In particular, for 4 steps, we picked a shift value of $\gamma = 5$ for t_{student}, outperforming the DMD2 baseline in [69]; see Table 11.

Method	shift t_{student}	NFE	Overall score	Quality score	Semantic score
DMD2 [69]		4	84.56	85.58	**80.50**
DMD2-*v*	10	4	84.53	85.95	78.84
DMD2-*v*	5	4	**84.60**	**86.03**	79.87
DMD2-*v*	10	3	**84.48**	**85.71**	79.58
DMD2-*v*	5	3	84.47	85.55	**80.15**

Table 11. VBench results of the DMD2 version in [69] and our DMD2-*v* for 3 and 4 steps with different timestep shifts for t_{student}.

B.7. Inference time

In Table Table 12, we measure the inference time of our proposed decoupled architecture to confirm that the flow head is lightweight compared to the backbone. Moreover, the gray columns validate the usage of *effective* NFE for evaluating efficiency, since the increase in the per-step inference time closely matches the effective NFE.

Setting	Backbone	Head	Fusion	Total	Increase	NFE
N4H5 (1.3B)	783	156	4.8	1426	1.51x	1.5
N2H5 (1.3B)	781	155	4.7	1100	1.17x	1.17
N4H5 (14B)	4191	595	29	6687	1.4x	1.38

Table 12. Inference time (in milliseconds) of different architecture components with $M = 1$ and batch-size 1 on a single H100 GPU. The increase in inference time (second-to-last column) is measured relative to a standard forward pass of the teacher model and closely matches the effective NFE (last column).

B.8. Training iterations

We note that TMD is training-efficient compared to other baselines. For instance, for Wan2.1 14B and $M = 2$ we

only require 3k iterations of lightweight TM-MF pretraining and 1k iterations of distribution matching, whereas rCM uses 10k iterations (with VSD and sCM losses) [69]. For Wan2.1 1.3B, we show in Table 13 that even with significantly reduced number of TM-MF iterations, TMD still outperforms rCM and DMD2-*v*.

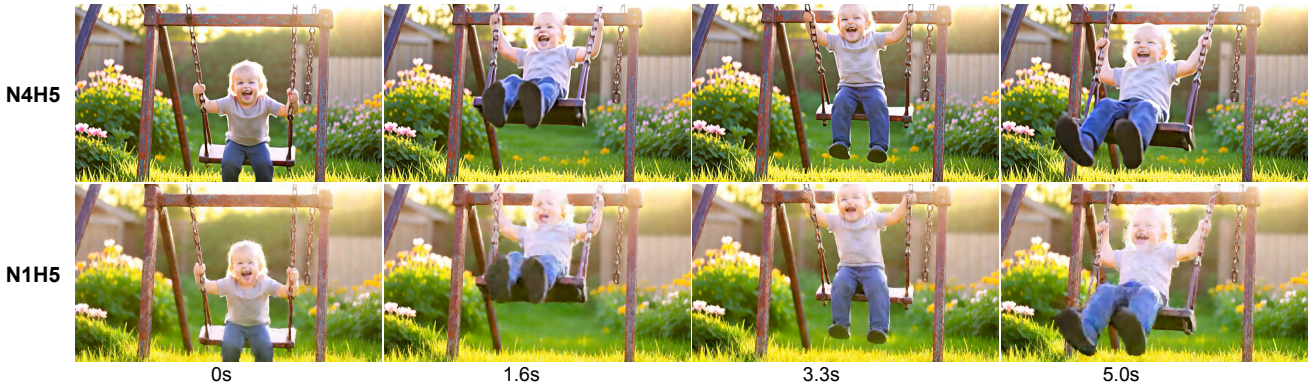
Method	TM-MF (iters)	Distill (iters)	Overall score	Quality score	Semantic score
rCM [69]	-	10k	84.09	84.90	**80.86**
DMD2-*v*	-	<6k	84.39	<u>85.65</u>	79.32
TMD-N2H5	0.5k	<6k	84.52	<u>85.65</u>	79.99
TMD-N2H5	1k	<6k	<u>84.59</u>	85.57	<u>80.68</u>
TMD-N2H5	3k	<6k	**84.68**	**85.71**	80.55

Table 13. Comparison of VBench scores for distilling Wan2.1 1.3B using TMD ($M = 2$) with a varying number of TM-MF pretraining iterations against two-step baselines.

B.9. More visual comparison results

We provide further visual comparisons between the 50-step teacher models with classifier-free guidance (CFG), DMD2-*v*, and TMD in Figures 14 to 19.

A joyful child, with a big smile and arms spread wide, swings energetically on a rusty old swing set in a sunlit backyard. The swing set, with peeling paint and creaking chains, contrasts against the vibrant green grass and blooming flowers surrounding it. The child's laughter echoes as they swing higher and higher, their feet barely touching the ground at the bottom of each arc.



Impressionist style, a single yellow rubber duck gently floating on undulating waves at sunset. The duck has a cheerful smile painted on its face, with sunlight casting warm, golden hues across the water. The sky is filled with soft, pastel shades of orange, pink, and purple, reflecting off the rippling surface. The water appears calm yet lively, with subtle waves carrying the duck along smoothly.

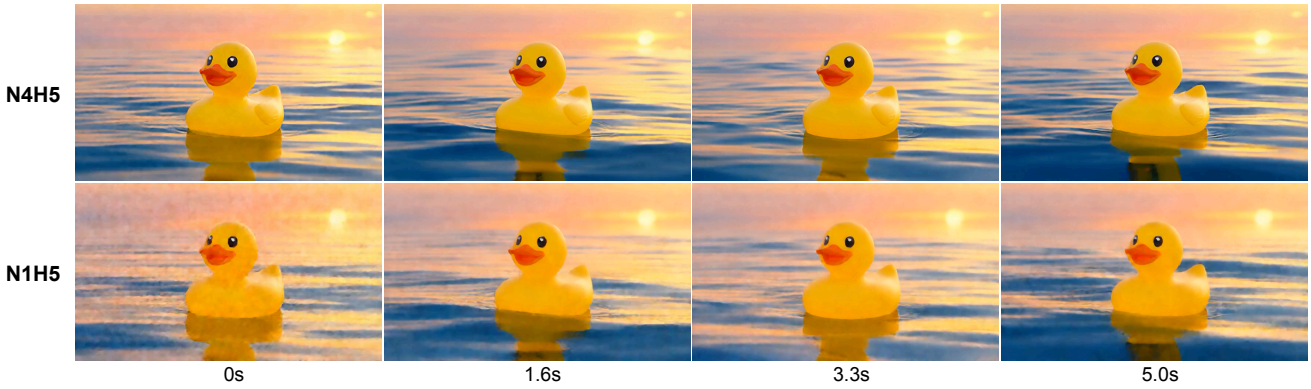


Figure 13. **Impact of flow head recurrence.** We show the impact of recurrence in the flow head by setting the number of flow head steps to 1 only at inference (*i.e.*, N1H5) when distilling Wan2.1 1.3B with $M = 2$ and the N4H5 setting for flow head (*i.e.*, 4 denoising steps and 5 DiT blocks in flow head). We observe that the videos generated without recurrence (marked by N1H5) are of much lower quality (*e.g.*, more artifacts and blurriness) than ones with recurrence (marked by N4H5), implying the importance of the fine-grained iterative refinement on our method.

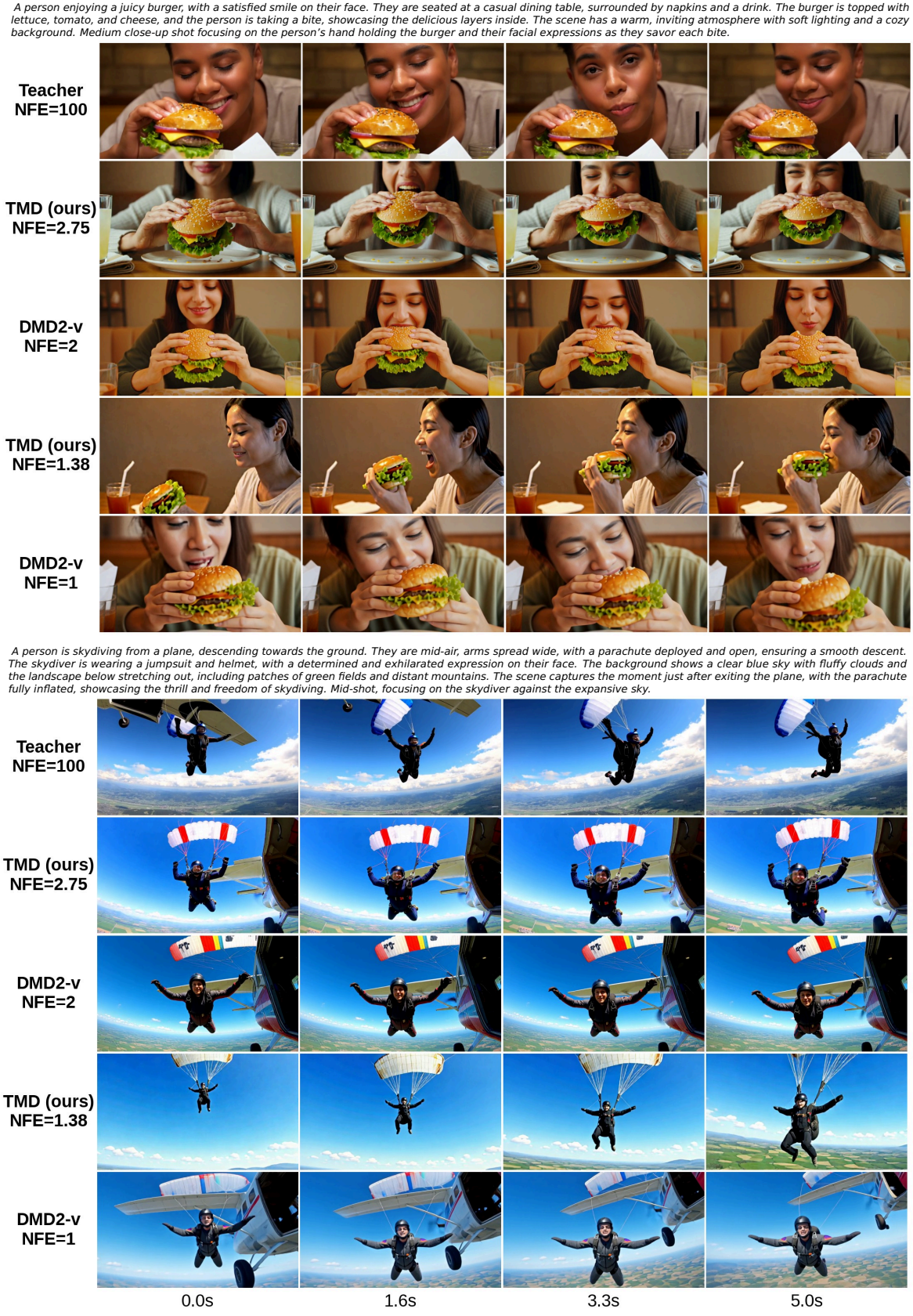


Figure 14. **Visual comparison on Wan2.1 14B.** We compare the outputs of the teacher, TMD, and DMD2-*v* on exemplary prompts.

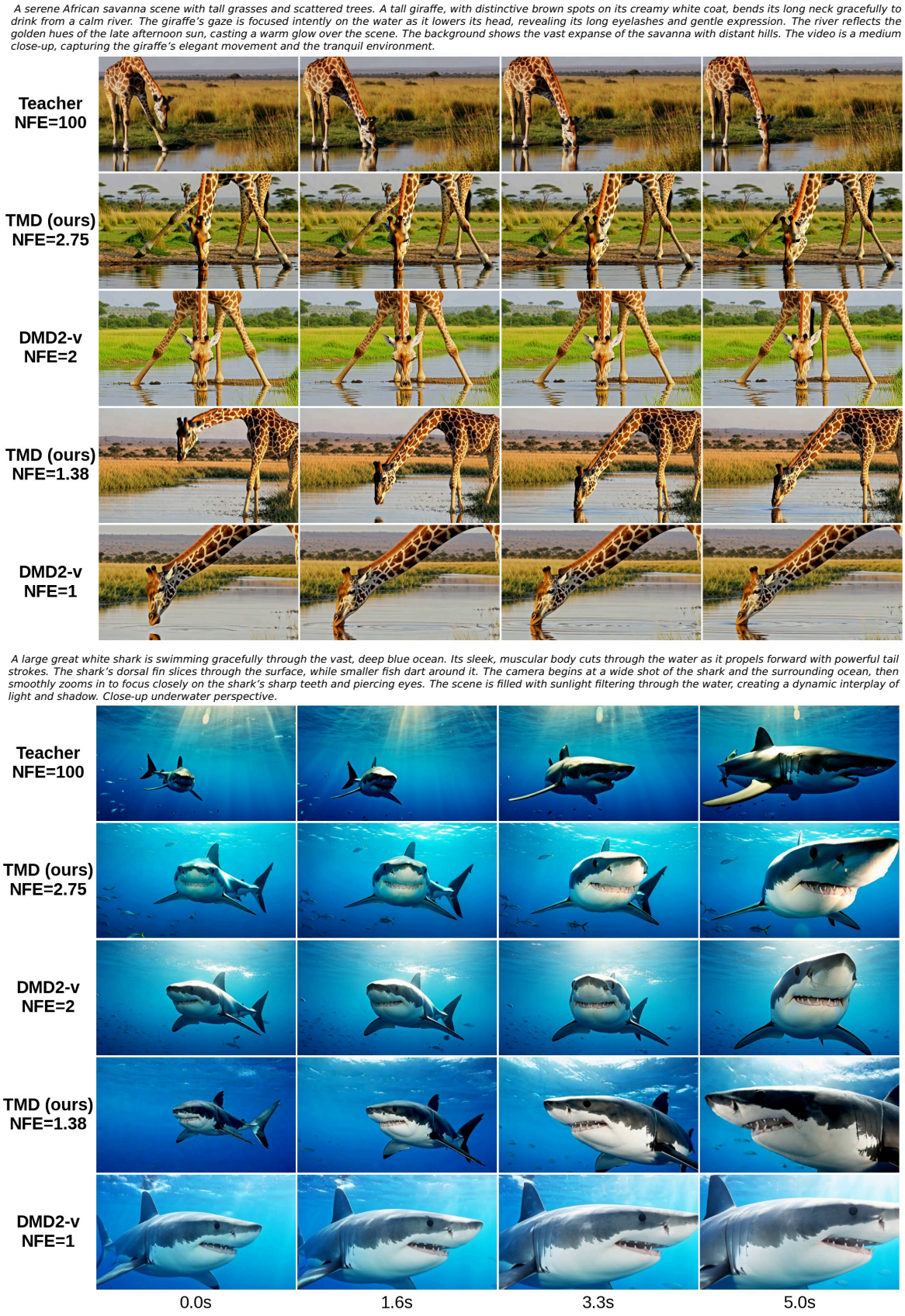


Figure 15. **Visual comparison on Wan2.1 14B.** We compare the outputs of the teacher, TMD, and DMD2-*v* on exemplary prompts.

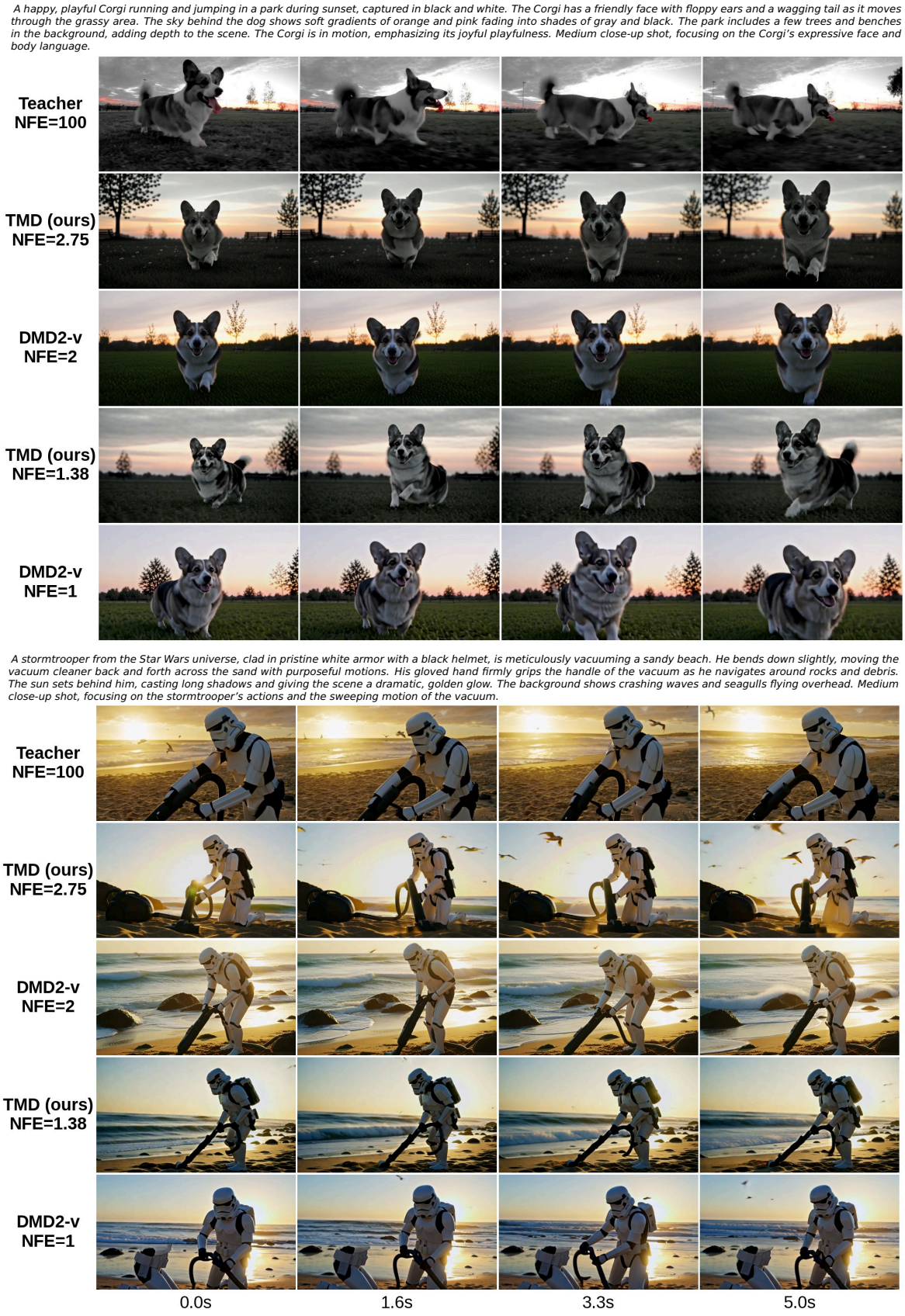


Figure 16. **Visual comparison on Wan2.1 14B.** We compare the outputs of the teacher, TMD, and DMD2-*v* on exemplary prompts.



Figure 17. **Visual comparison on Wan2.1 1.3B.** We compare the outputs of the teacher, TMD, and DMD2-*v* on exemplary prompts.

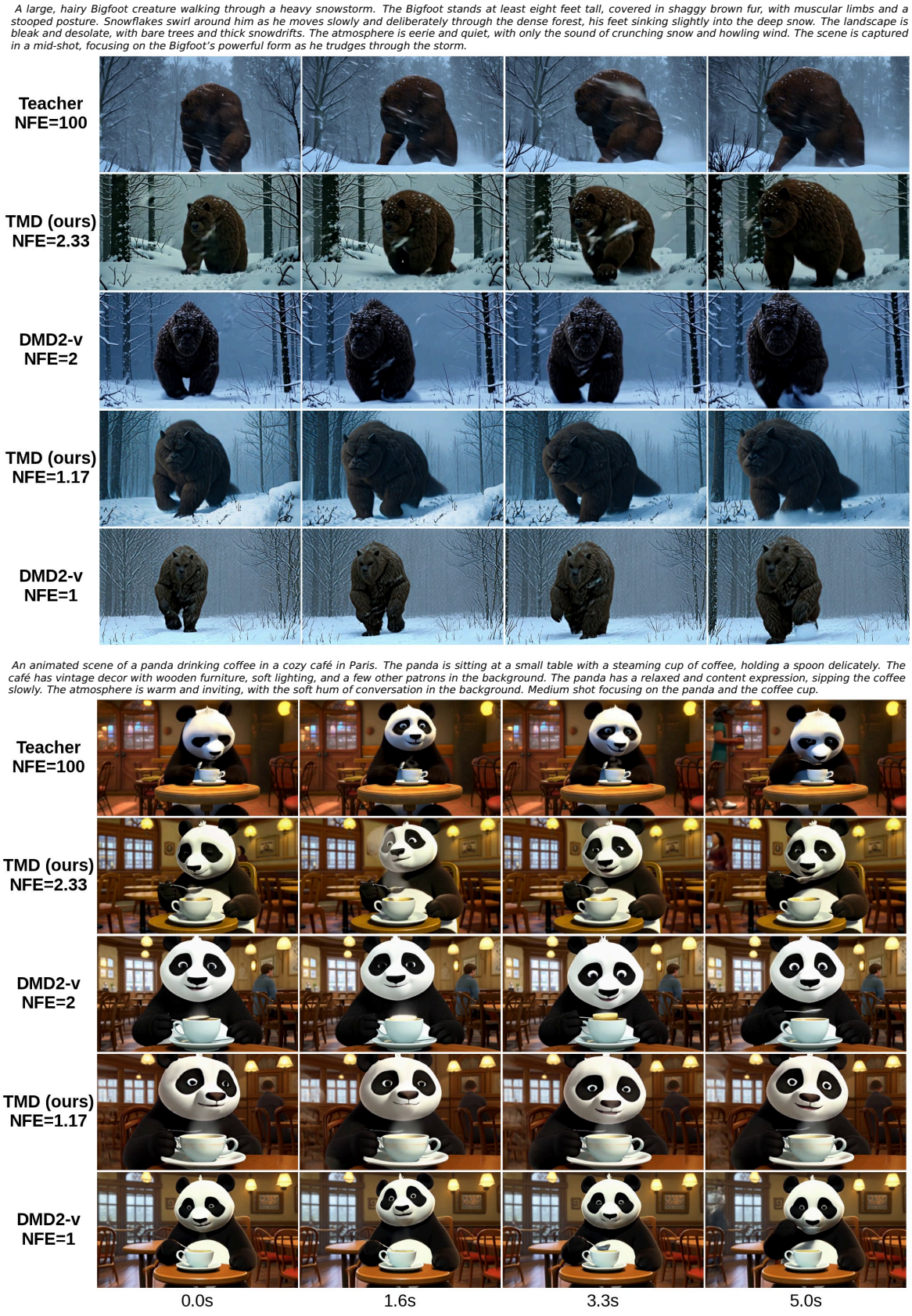


Figure 18. **Visual comparison on Wan2.1 1.3B.** We compare the outputs of the teacher, TMD, and DMD2-*v* on exemplary prompts.

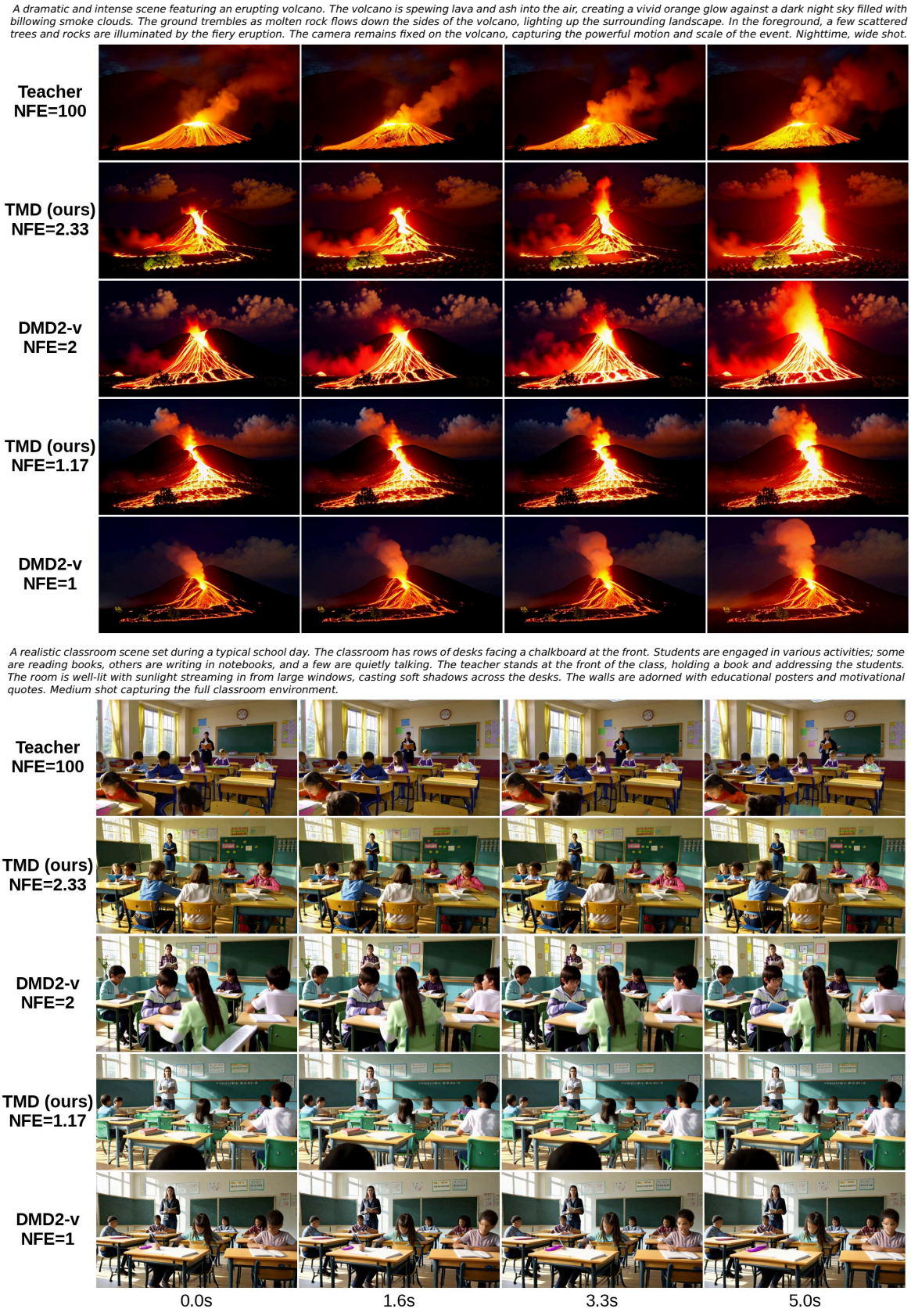


Figure 19. **Visual comparison on Wan2.1 1.3B.** We compare the outputs of the teacher, TMD, and DMD2-*v* on exemplary prompts.

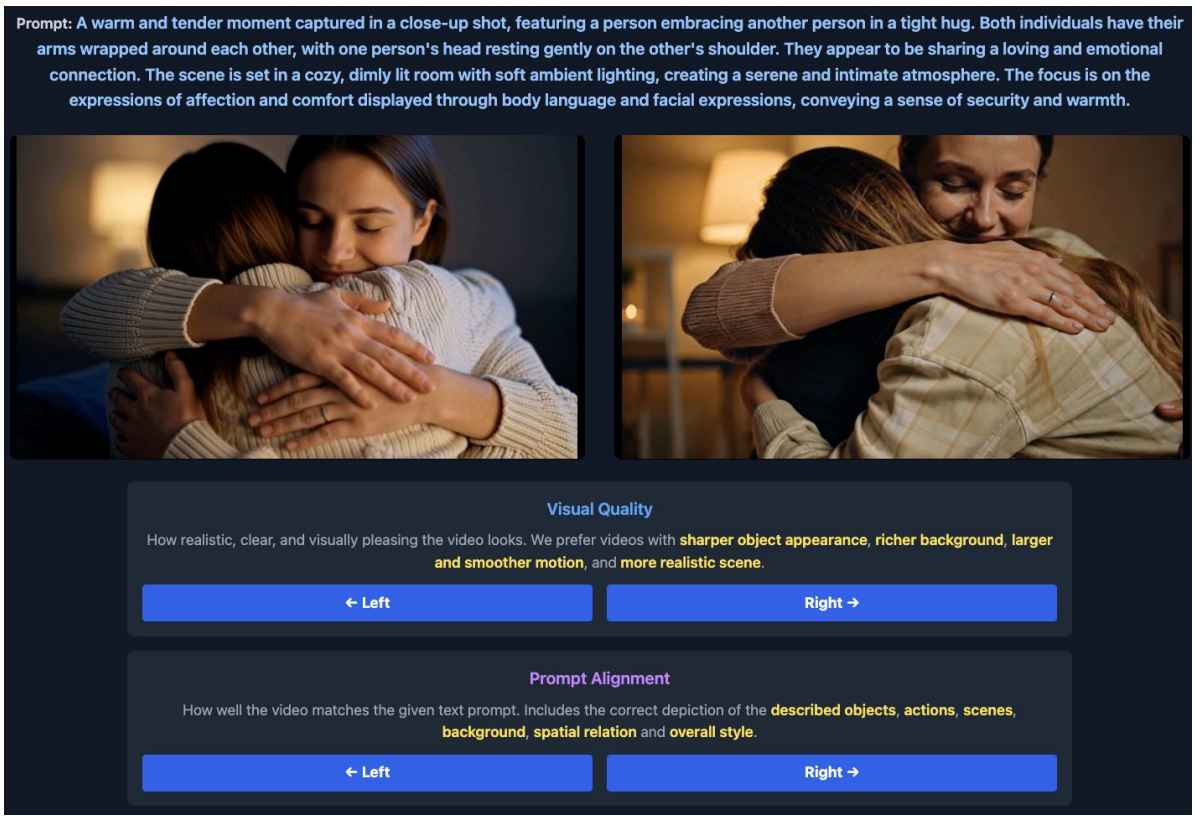


Figure 20. **User study interface.** Screenshot of our user preference study interface explained in Section 4.2.